Fundamentals of Recursion

Lecture Flow

1. Prerequisites
2. Motivation Problem
3. Basic Concepts
4. Time and Space Complexity
5. Common Pitfalls
6. Practice Questions and Resources
7. Quote of the Day

Prerequisites

- Understanding iterative program flow
- Functions, local variables and global variables
- Stack data structures
- Willingness to learn

Motivation Problem

Imagine you are in a long queue and the reception asks you how many people there are in the line.

Each person can only talk to the person behind and infront of them.

How do you count the number of people in the queue?

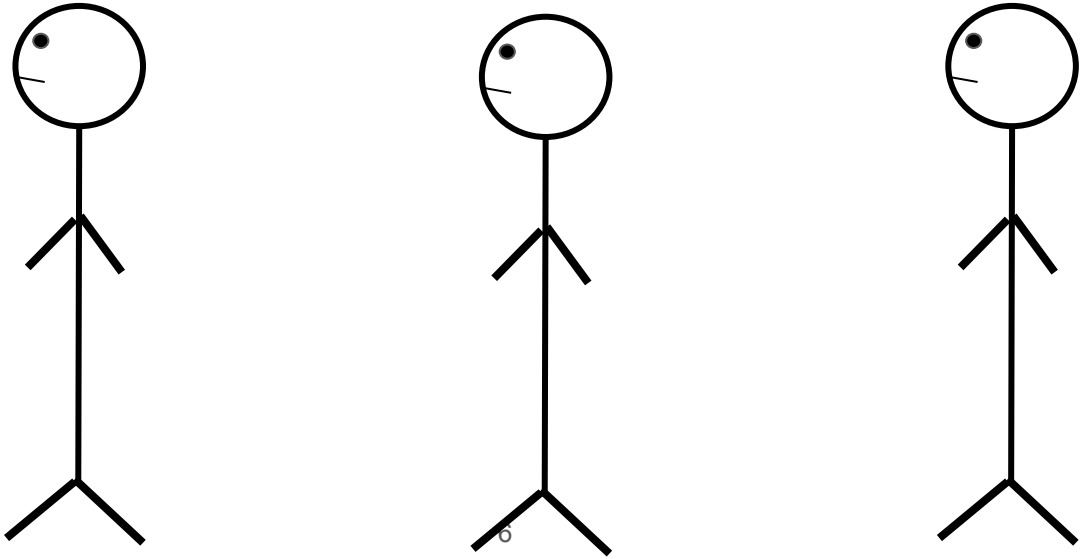
Brainstorm

- Let's assume each person in the queue
  - Can increment numbers by 1
  - Can request other people
- What if I asked the person behind me "How many people are there behind you ?" and whatever their response is, I can add myself and do a plus 1 and inform the reception? And
- What if the person I asked, asks the person behind him "How many people are there behind you ?" and whatever he gets, he adds himself and do plus 1 and informs me?

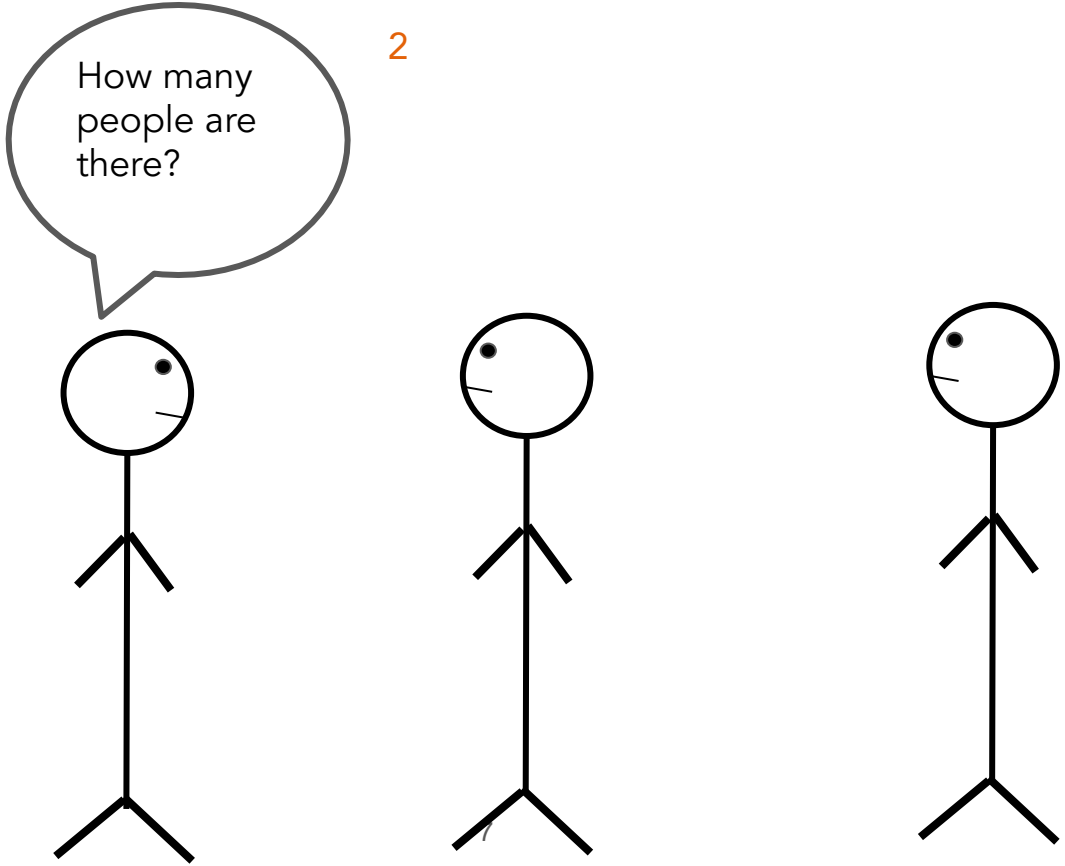
⋮

1

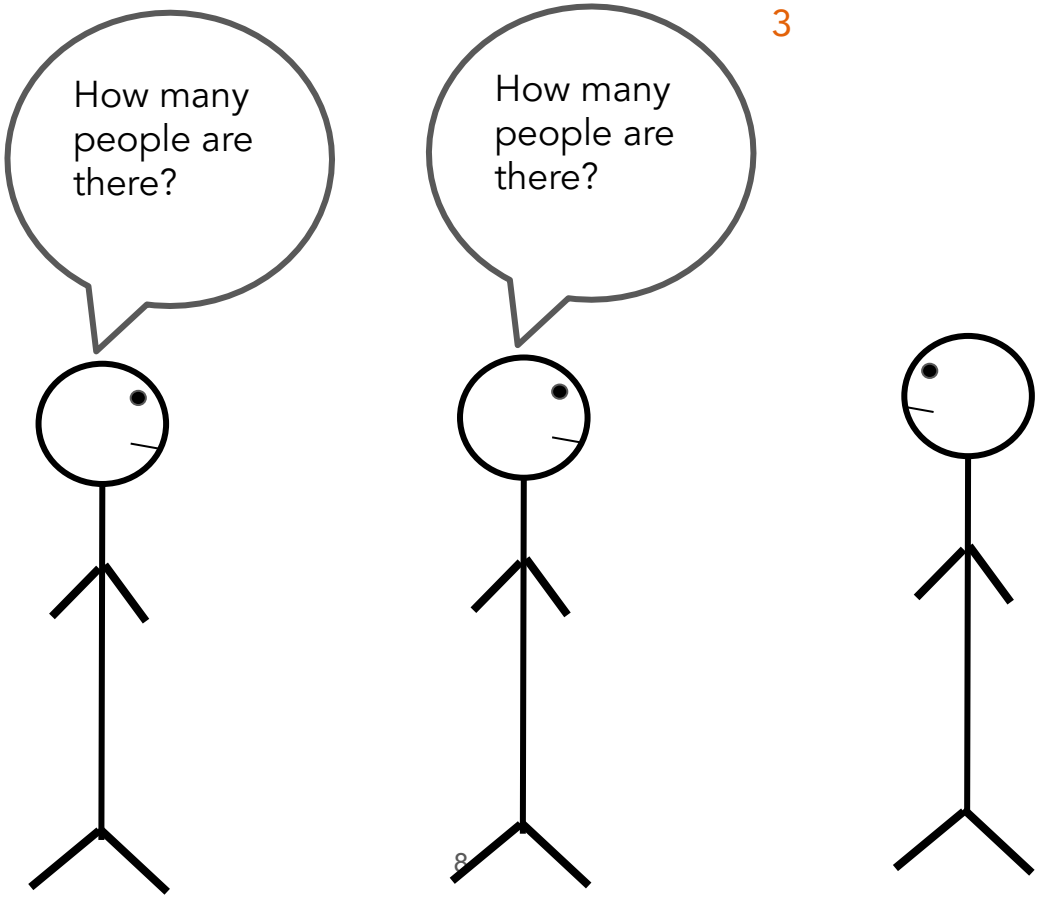
How many people are there?



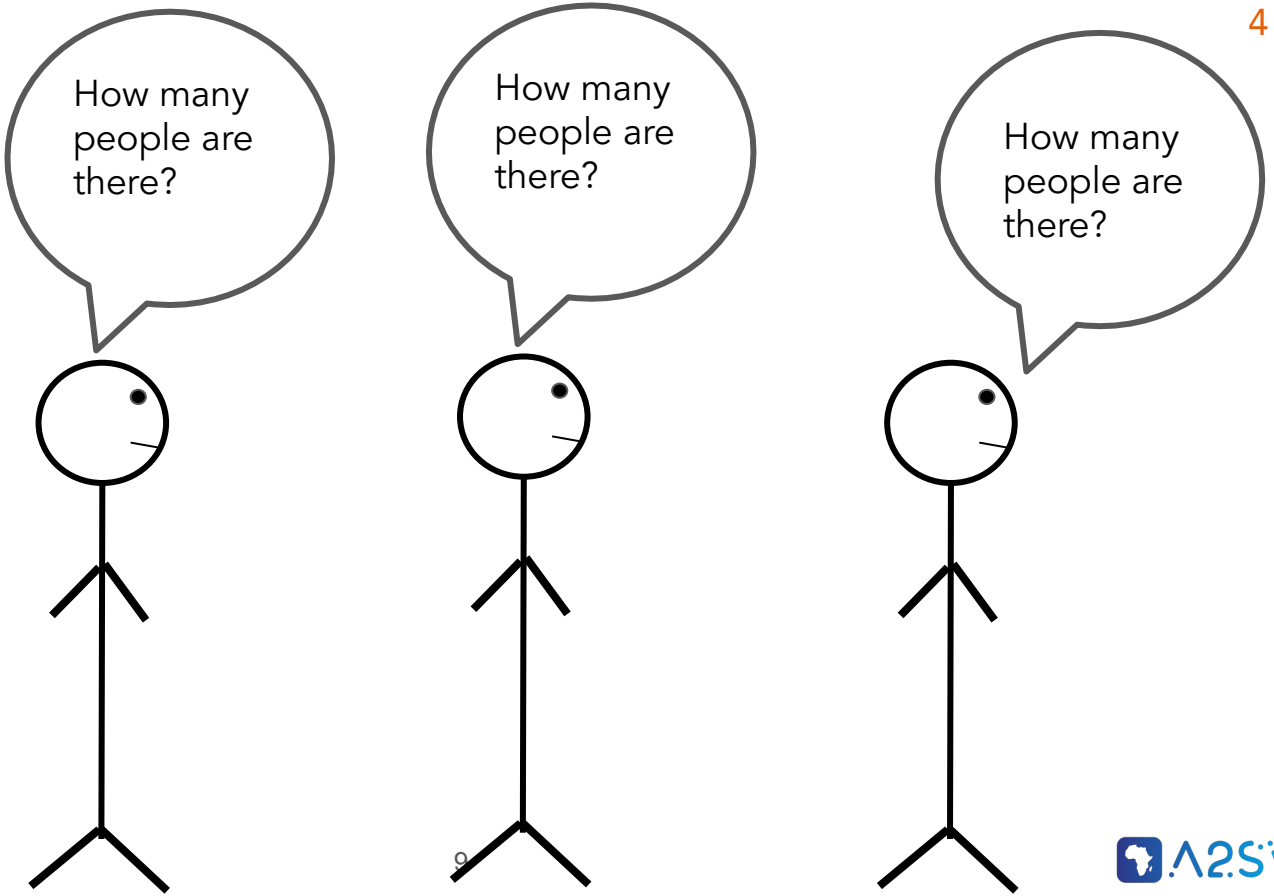
How many people are there?


How many people are there?
2

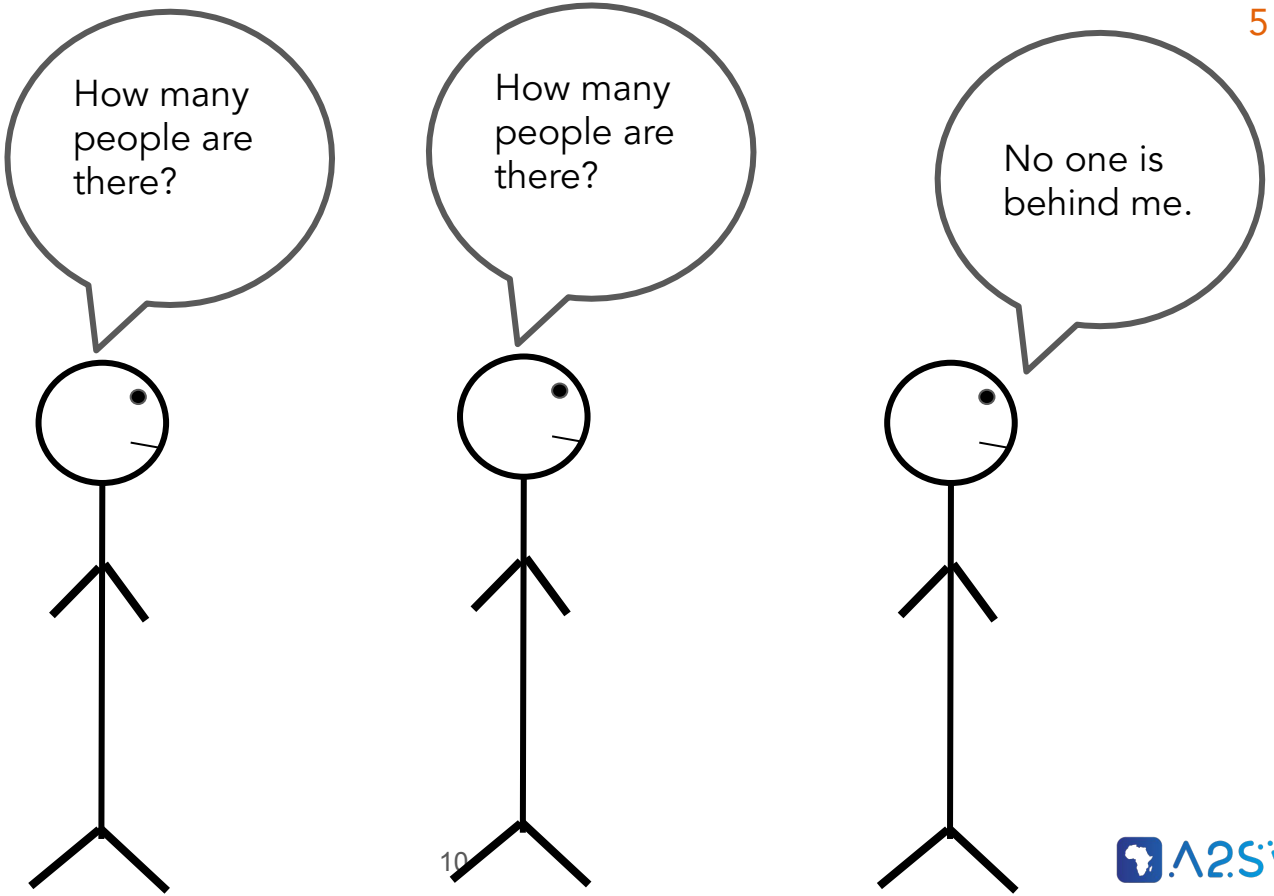
How many people are there?


How many people are there?
How many people are there?
3

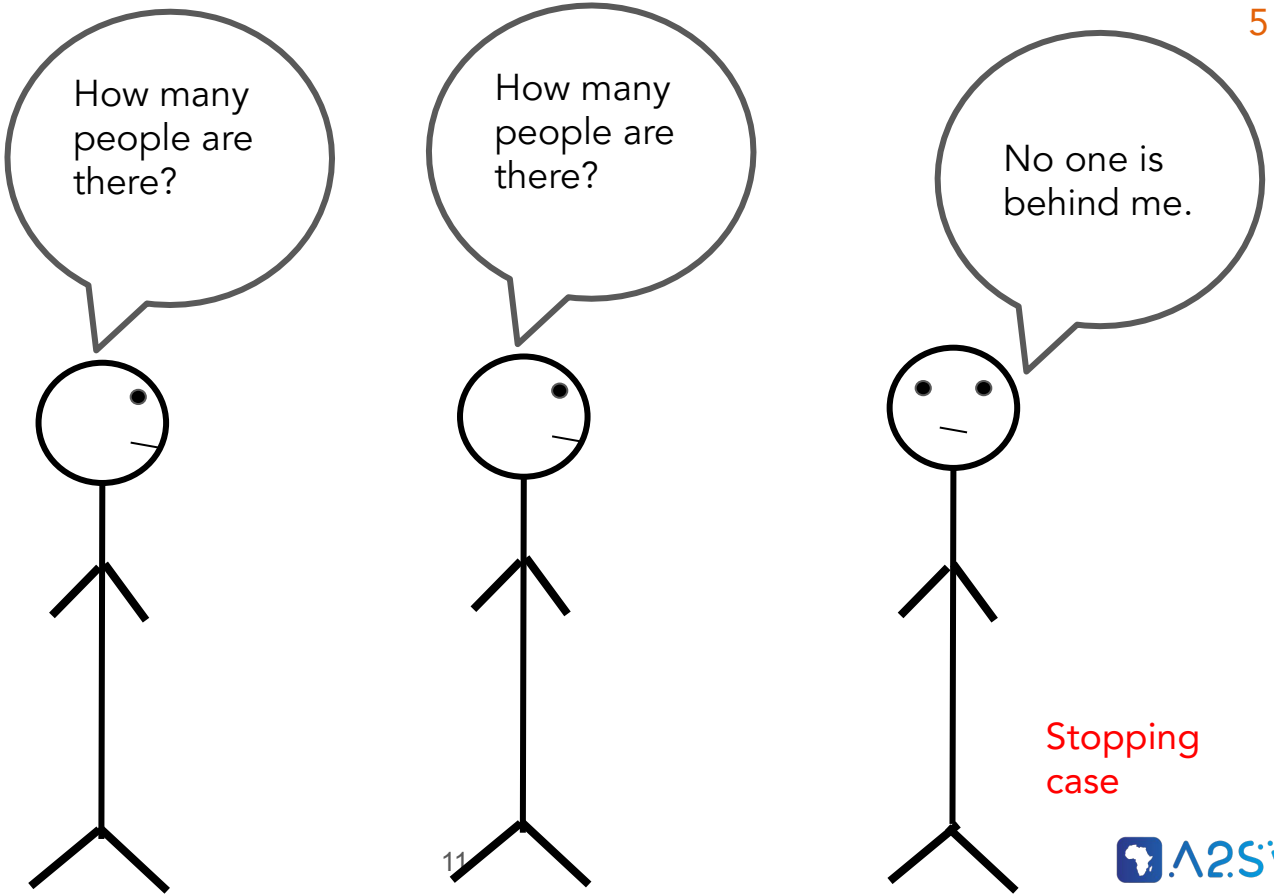
How many people are there?


How many people are there?
How many people are there?
How many people are there?

How many people are there?


How many people are there?
How many people are there?
No one is behind me.

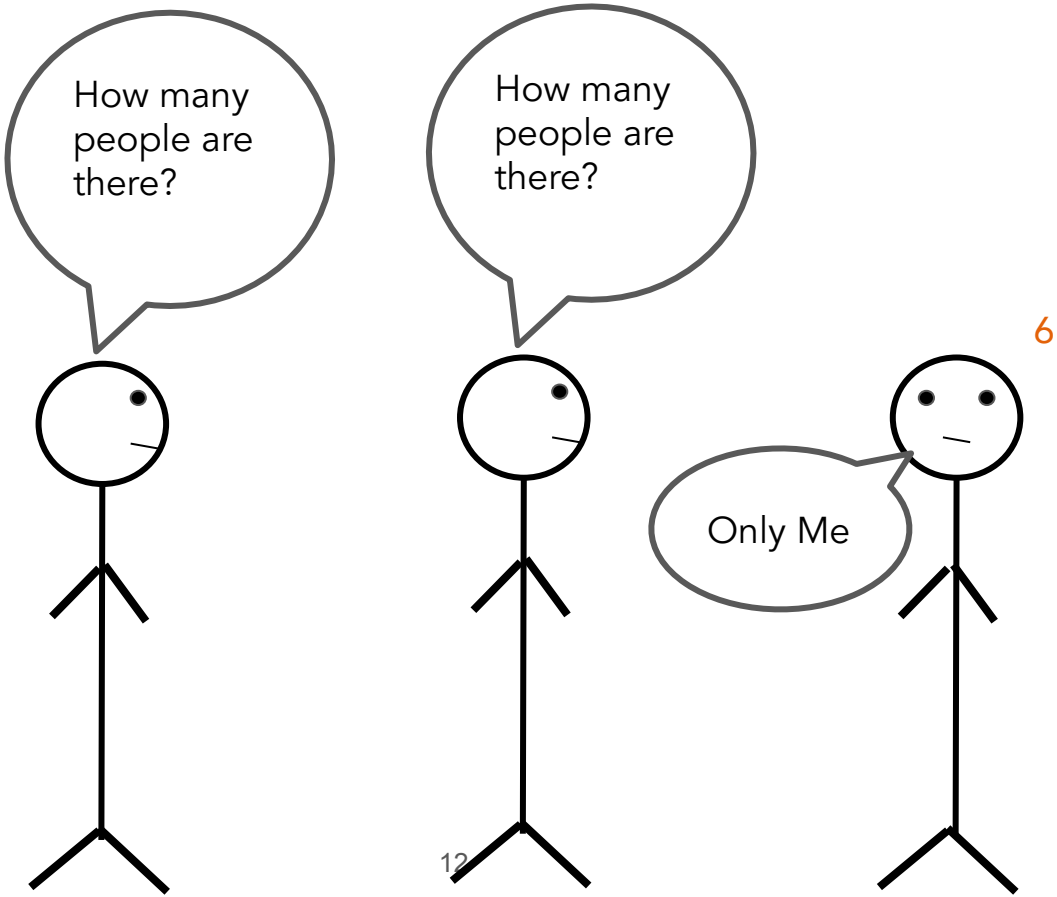
5



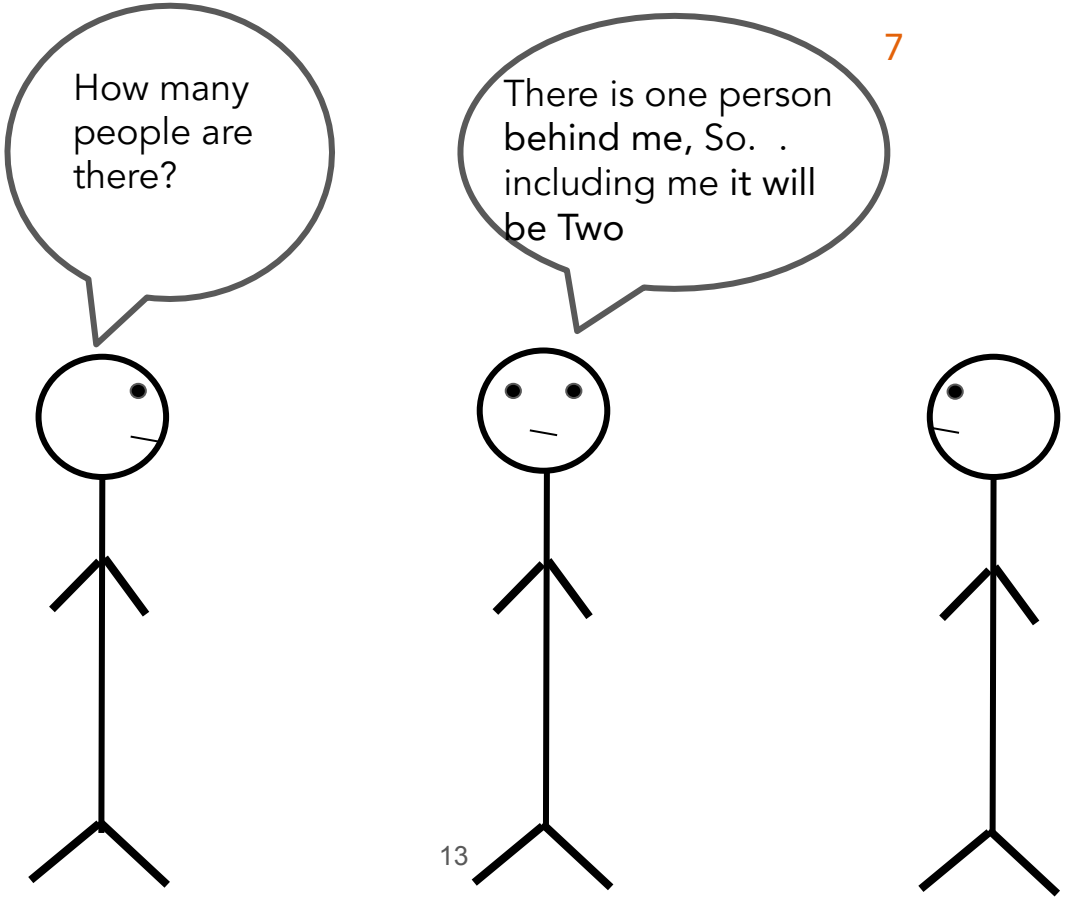
5

Stopping case

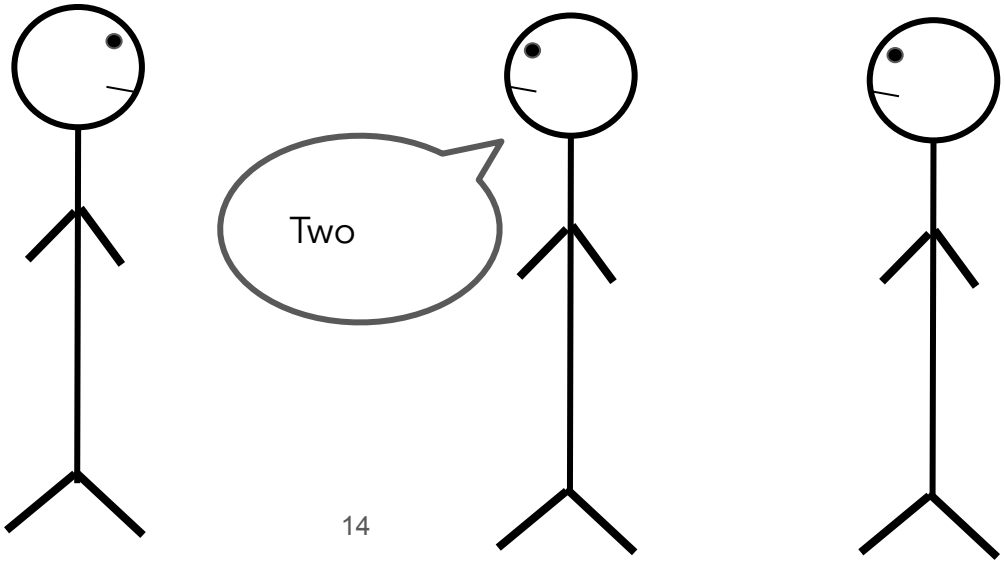
How many people are there?


How many people are there?
How many people are there?
6
Only Me

How many people are there?


How many people are there?
There is one person behind me, So. . including me it will be Two
7

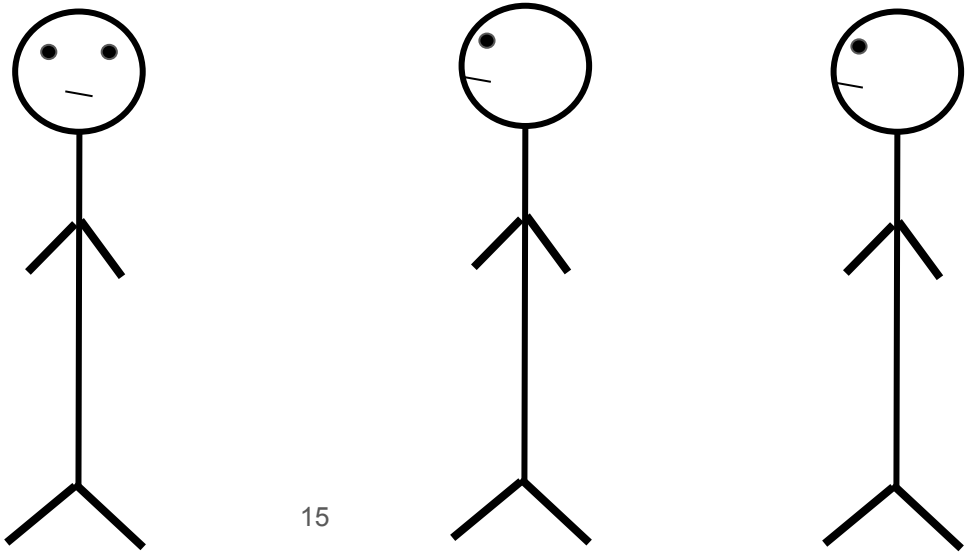
8



How many people are there?

9

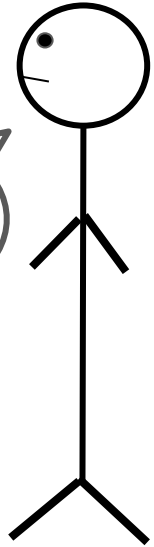
There are two people behind me, So. . including me it will be Three

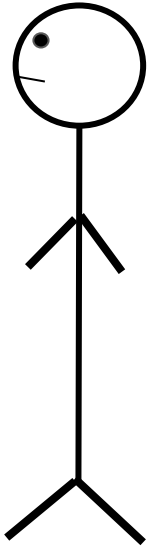


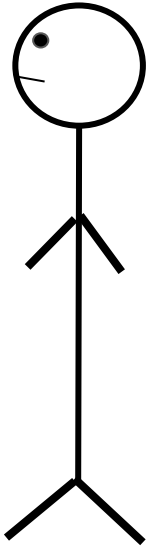
How many people are there?

Three

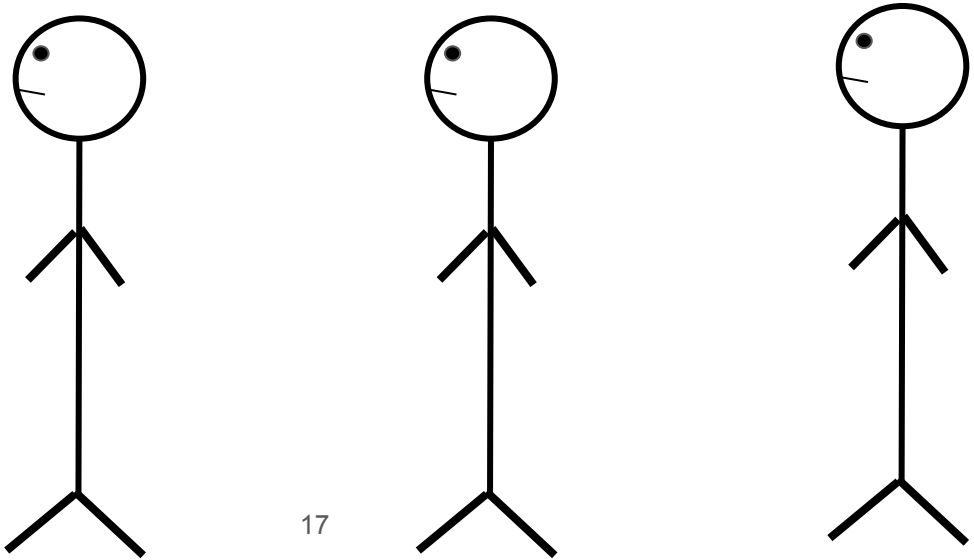
10







11



Can we simulate this with code?

- Since we are doing the same thing again and again let's use one function to do the task

Does this work?

```
def ask(person):

    behind = ask(person + 1)
    people = behind + 1

    return people
```

What happens?

```
behind =
ask(person + 1)
people = behind +
1

return people
```

```
behind =
ask(person + 1)
people = behind +
1

return people
```

```
behind = ask(person
+ 1)
people = behind + 1

return people
```

We will never get to return because we are calling the function non stop. What should we do?

Does this work?

```
def ask(person):
    if noMorePeople:
        return 1

    behind = ask(person + 1)
    people = behind + 1

    return people
```

What happens?

```
if noMorePeople:
    return 1

behind = ask(person + 1)
people = behind + 1

return people
```

```
if noMorePeople:
    return 1

behind = ask(person + 1)
people = behind + 1

return people
```

```
if noMorePeople:
    return 1

behind = ask(person + 1)
people = behind + 1

return people
```

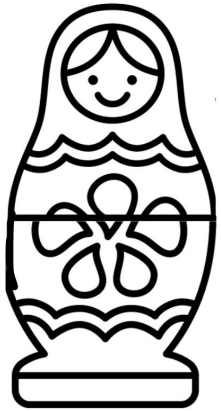
```
if noMorePeople:
    return 1

behind = ask(person + 1)
people = behind + 1

return people
```

Then now we return the number of people to our first caller

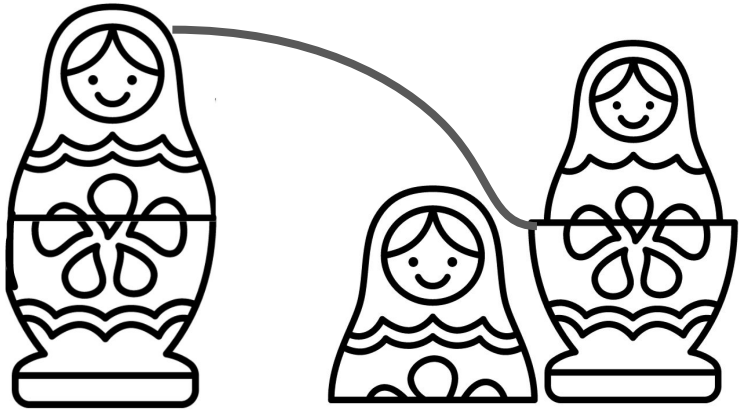
Real Life Example 2

Imagine some one give you a matryoshka and ask you to count the number of dolls, how would you solve the problem?

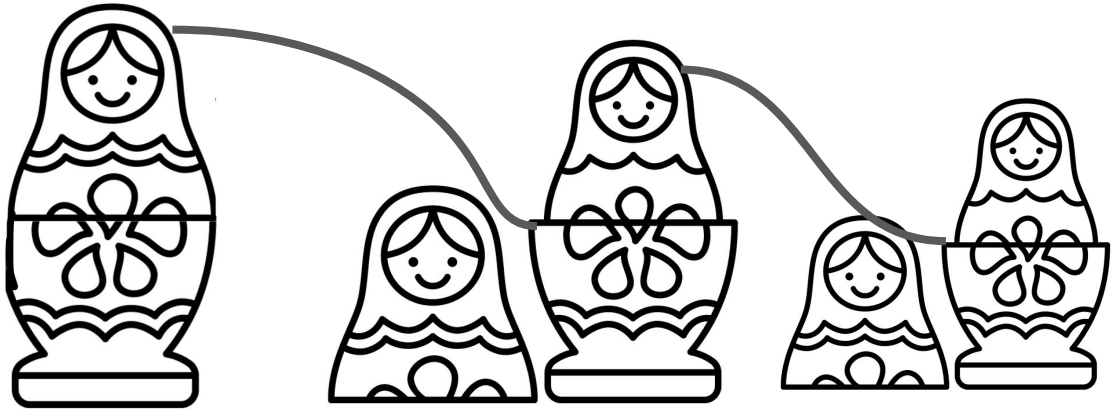
Brainstorm



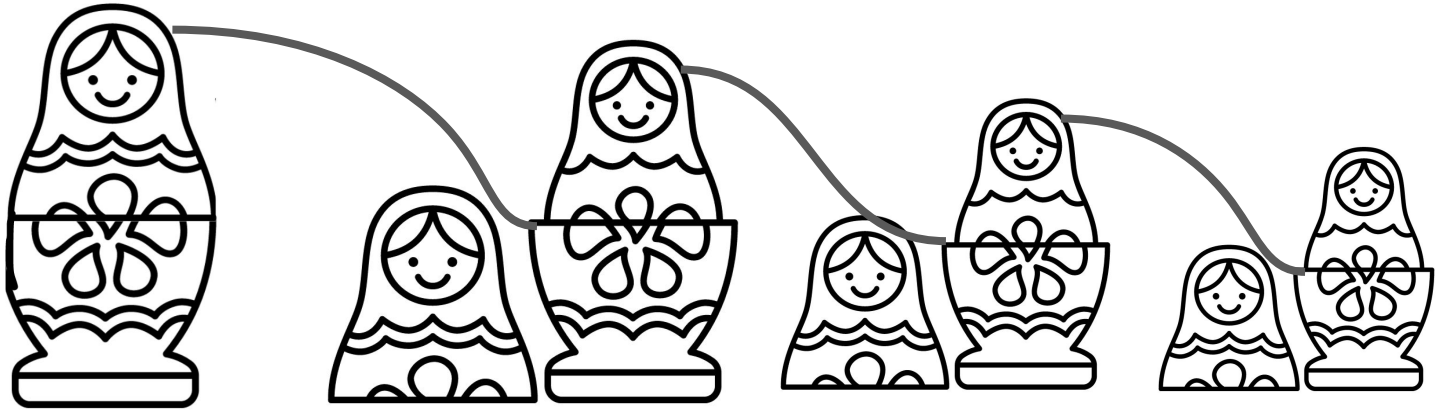
Brainstorm



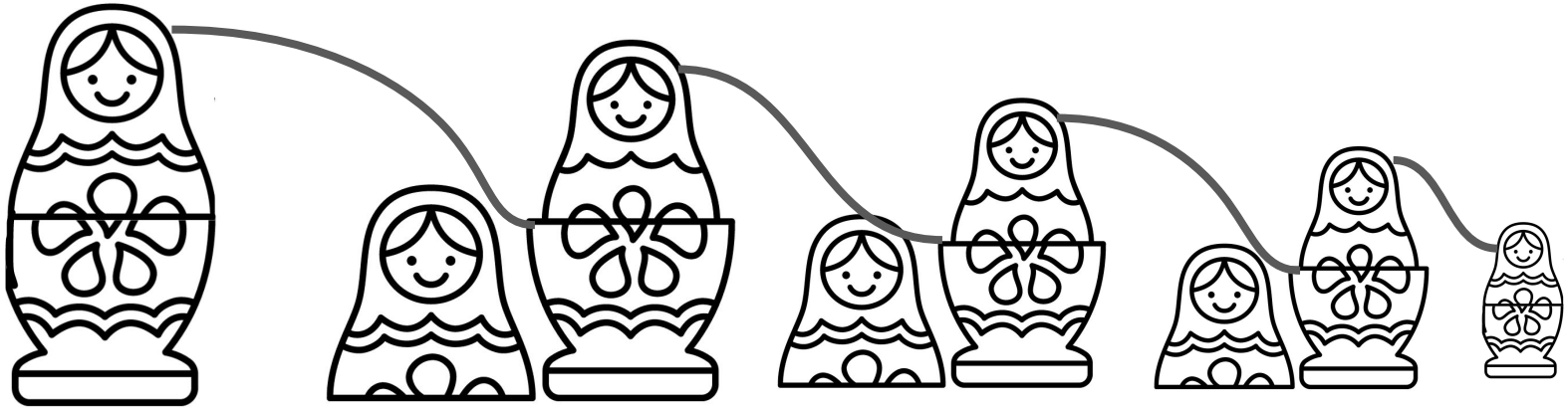
Brainstorm



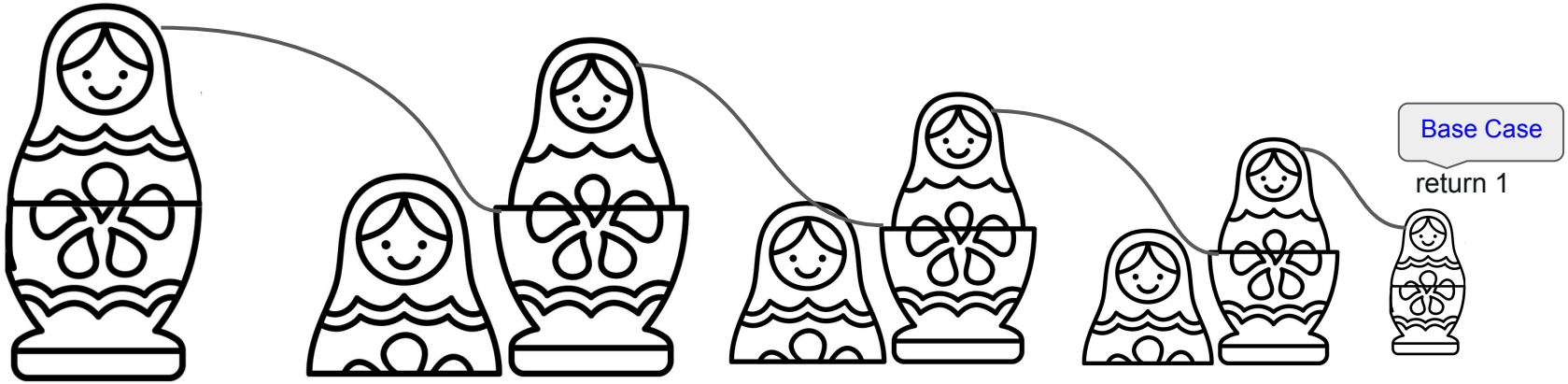
Brainstorm



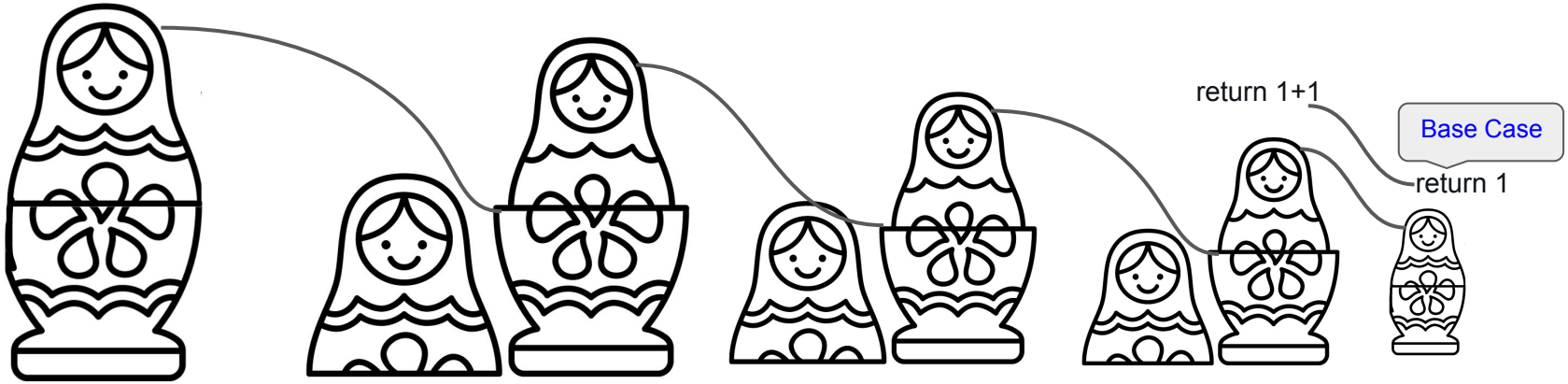
Brainstorm



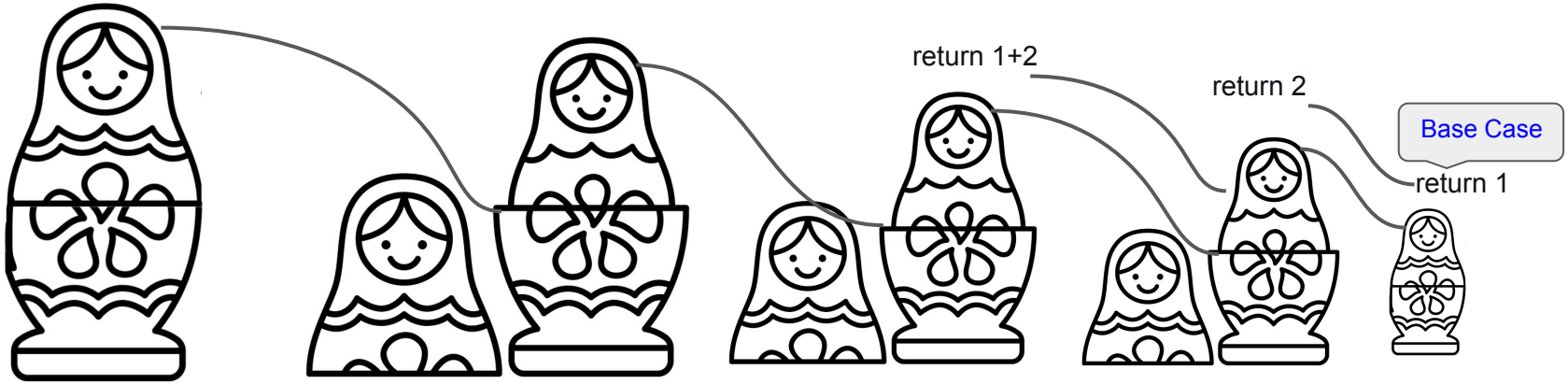
Brainstorm



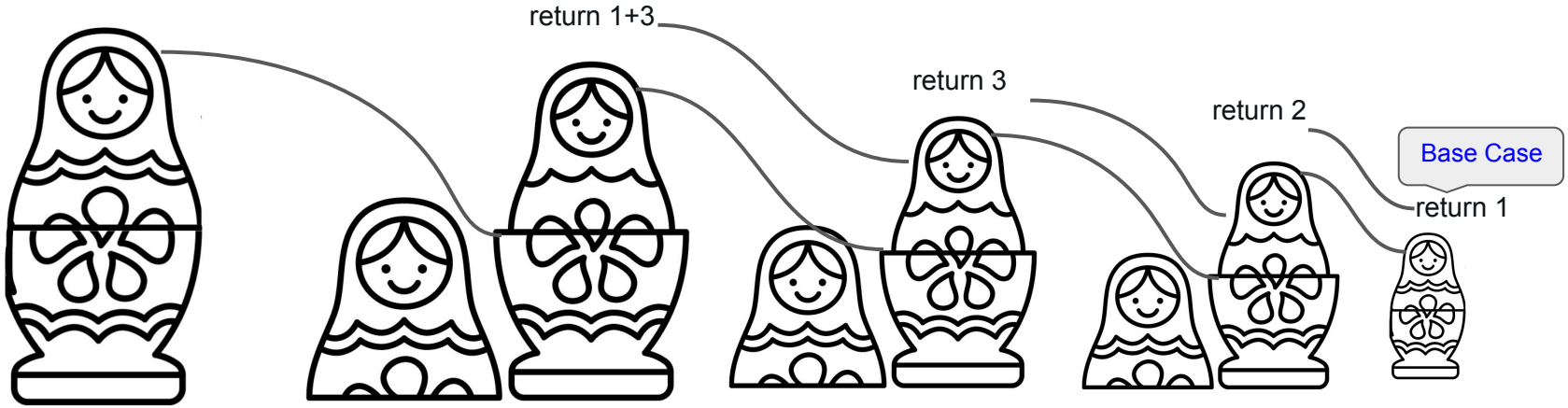
Brainstorm



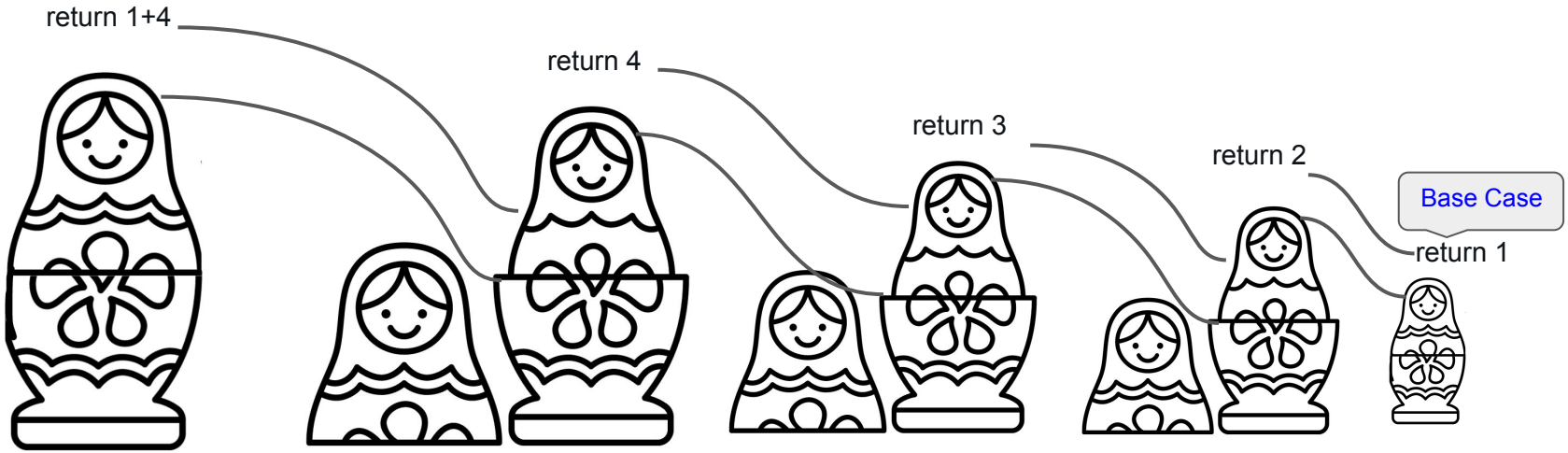
Brainstorm



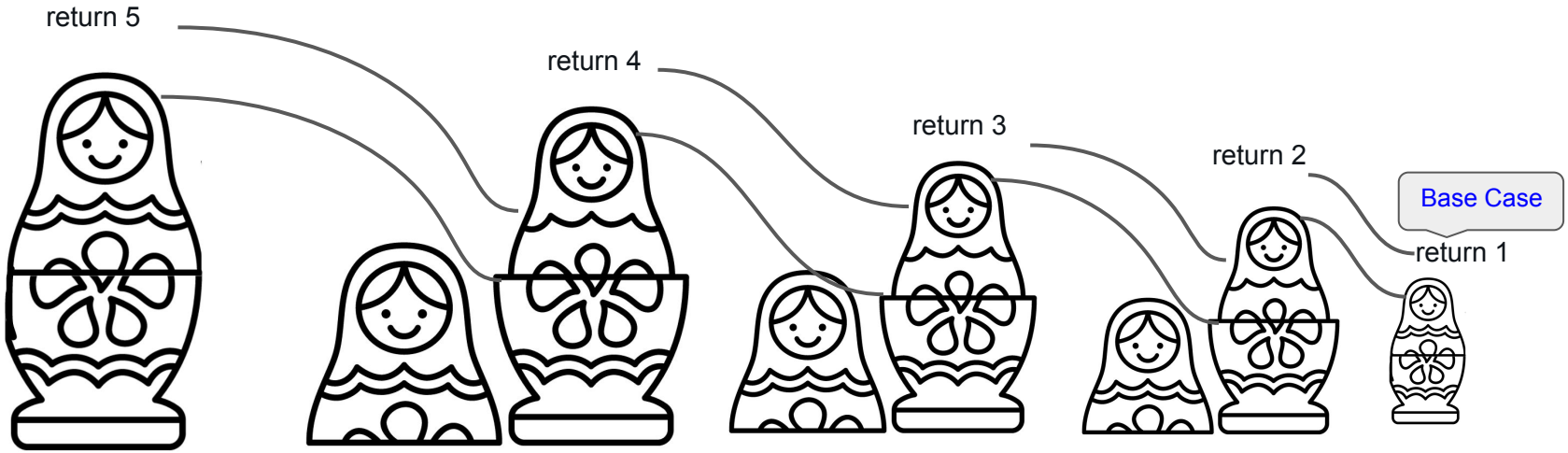
Brainstorm



Brainstorm



Brainstorm



Can we simulate this with code?

- Since we are doing the same thing again and again let's use one function to do the task

Does this work?

```
def count(matryoshka):
    inside = count(matryoshka.child)
    return inside + 1
```

What happens?

```
inside = count(matryoshka.child)
return inside + 1
```

```
inside = count(matryoshka.child)
return inside + 1
```

```
inside = count(matryoshka.child)
return inside + 1
```

...

We will never get to return because we are calling the function non stop. What should we do?

How about this?

```
def count(matryoshka):
    if matryoshka.empty:
        return 1

    inside = count(matryoshka.child)

    return inside + 1
```

What happens?

```
if matryoshka.empty:
        return 1

inside = count(matryoshka.child)
return inside + 1
```

```
if matryoshka.empty:
        return 1

inside = count(matryoshka.child)
return inside + 1
```

```
if matryoshka.empty:
        return 1

inside = count(matryoshka.child)
return inside + 1
```

Then now we return the number of people to our first caller

Definitions

Recursion: process in which a function calls itself directly

Basic Concept

A problem can be solved intuitively and much more easily if it is represented as an operation of it's smaller versions.

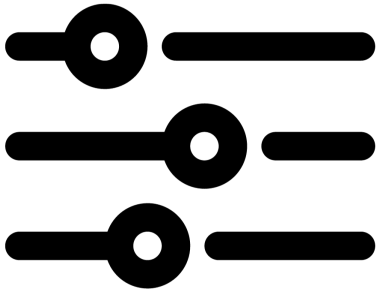
What was the problem in the previous real life example?

What was the subproblem?

What is the operation we perform ?

Basic components of recursion

- State
- Base case
- Recurrence relation

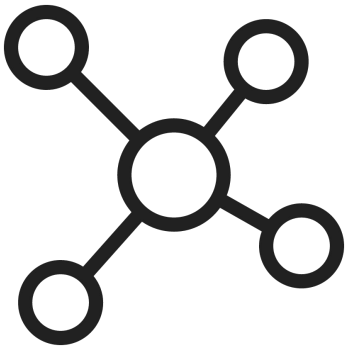


State

- The set of parameters that fully and *uniquely* describe a particular subproblem completely.

Base Case

- Is a known and the simplest state of the problem.
- Is a terminating scenario and can no longer be solved recursively.
- Initiates the process of returning to the original call function (or problem).



Recurrence Relation

- Defines the relation between subsequent states.
- It aggregates the result from recursive calls and returns the answer we have so far to the caller.
- This is a section where the function calls itself with a modified state.
- Changes in state should lead towards base cases. (i.e Should be a breakdown of the current problem into a smaller problem).

What is the state, the base case and recurrence relation for the previous problem?

State

```
def count(matryoshka):
    if matryoshka.empty:
        return 1

    prev = count(matryoshka.child)
    curr = prev + 1

    return curr
```

Base Case

Recurrence Relation

Practice

Factorial Trailing Zeros

What is the state, the base case and recurrence relation for the above problem?

Base Case

- $n == 1$

State

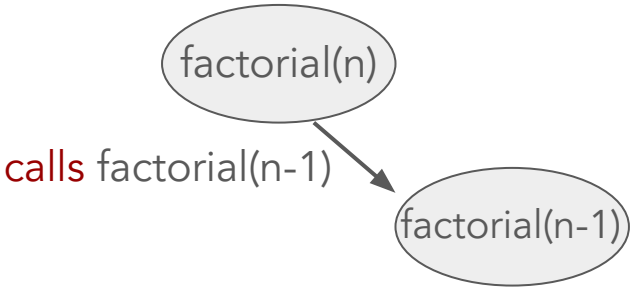
- n

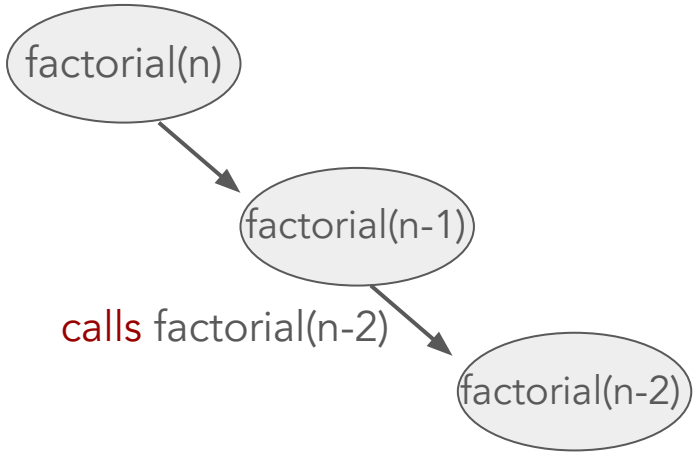
Recurrence Relation

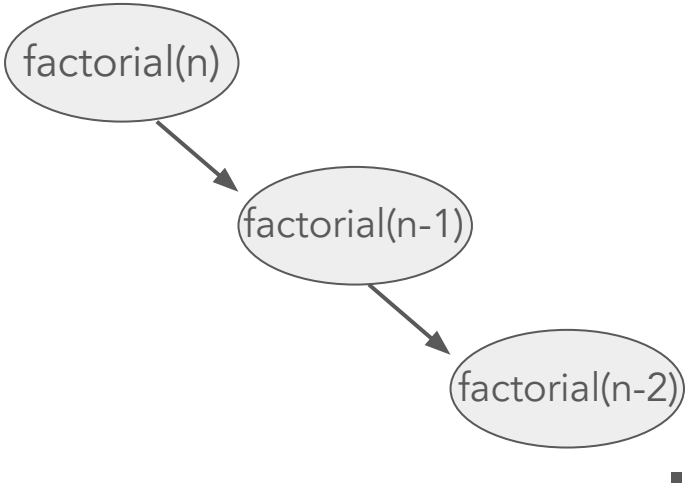
- $f(n) = n * f(n - 1)$

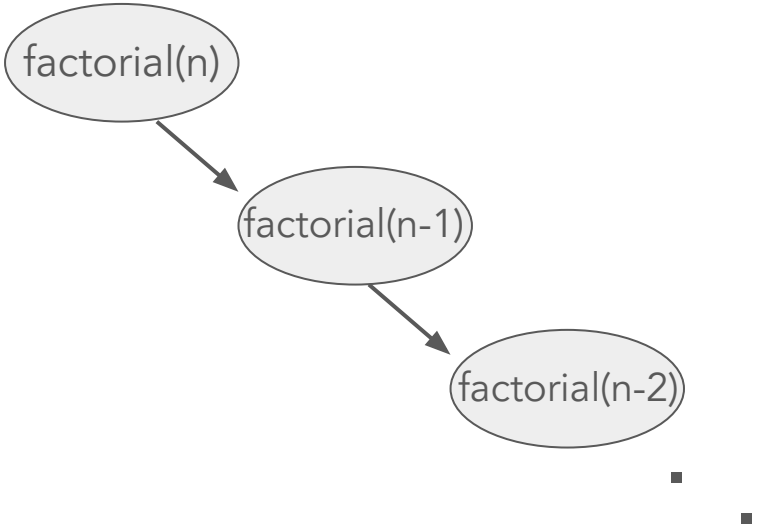
How does this work?

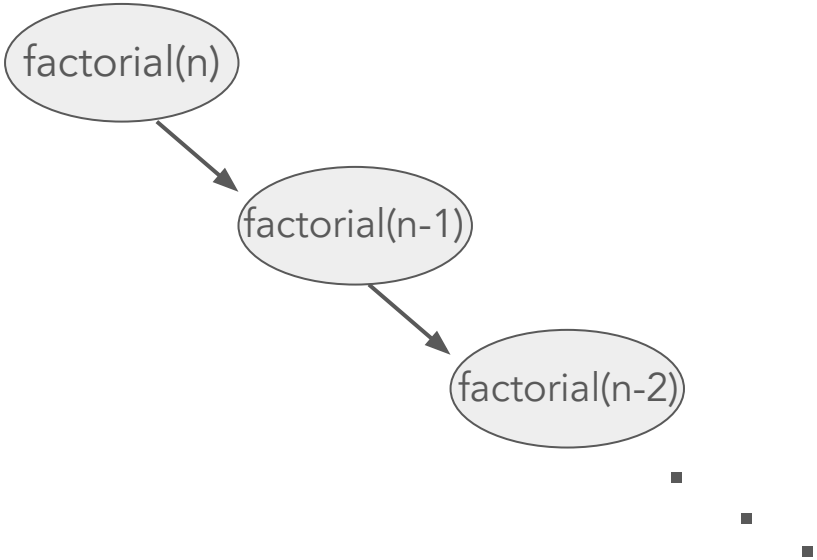
factorial(n) factorial of n gets called

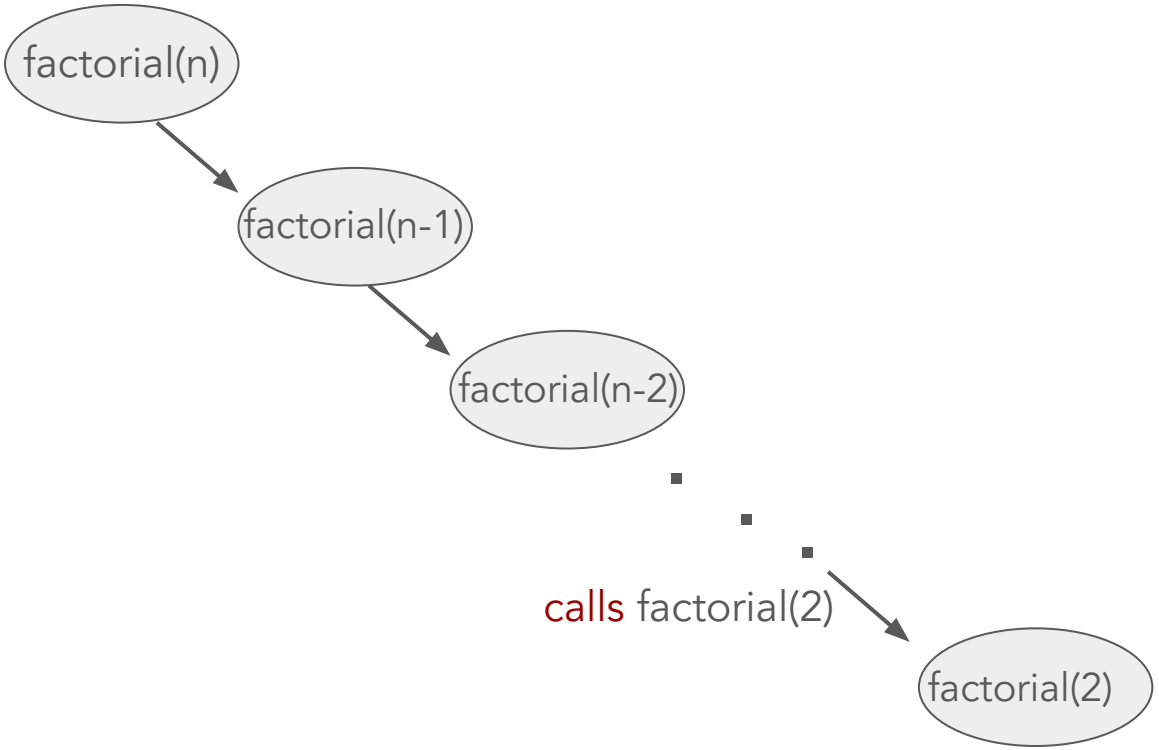

factorial(n)
calls factorial(n-1)
factorial(n-1)

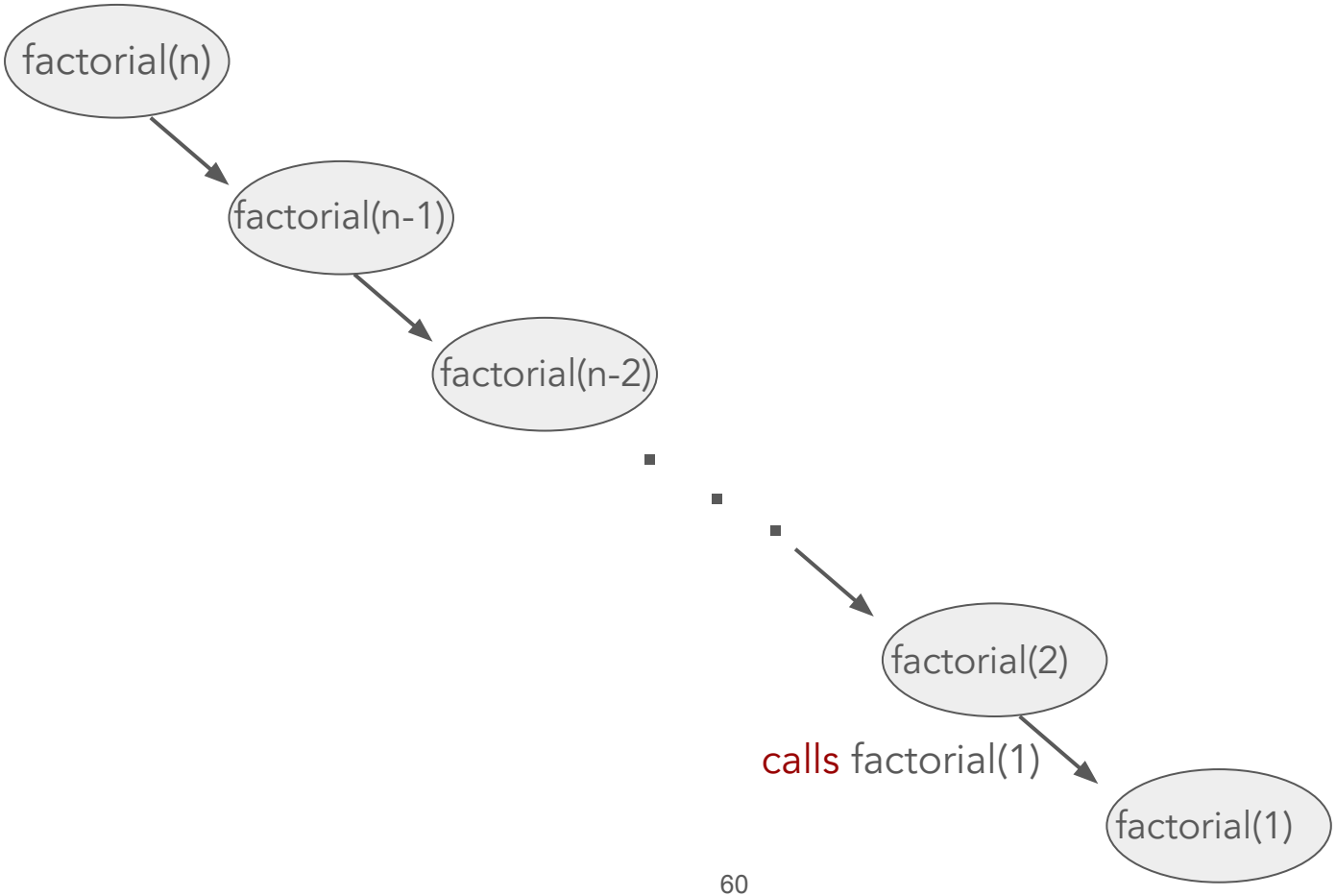

factorial(n)
factorial(n-1)
calls factorial(n-2)
factorial(n-2)

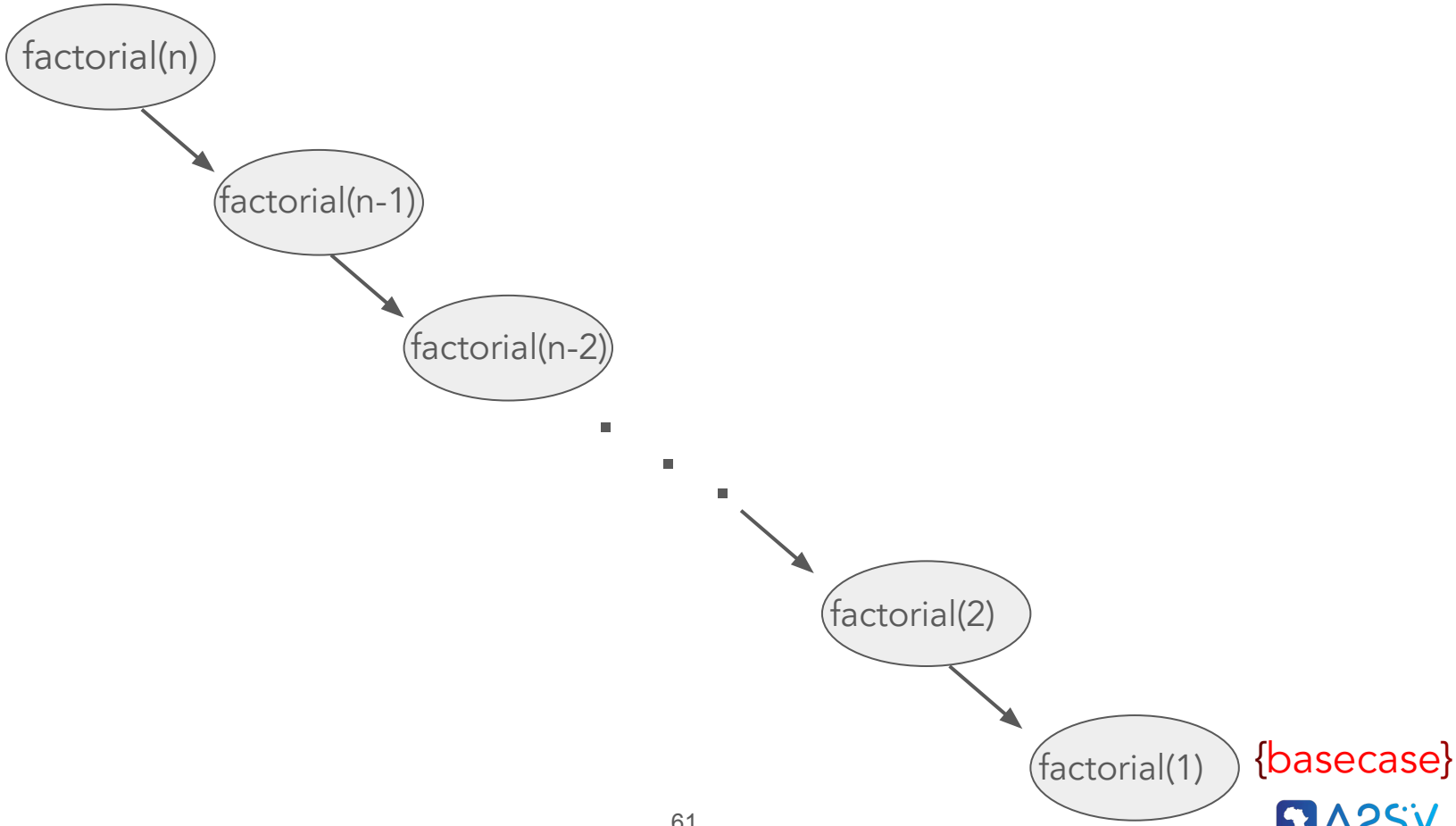

factorial(n)
factorial(n-1)
factorial(n-2)

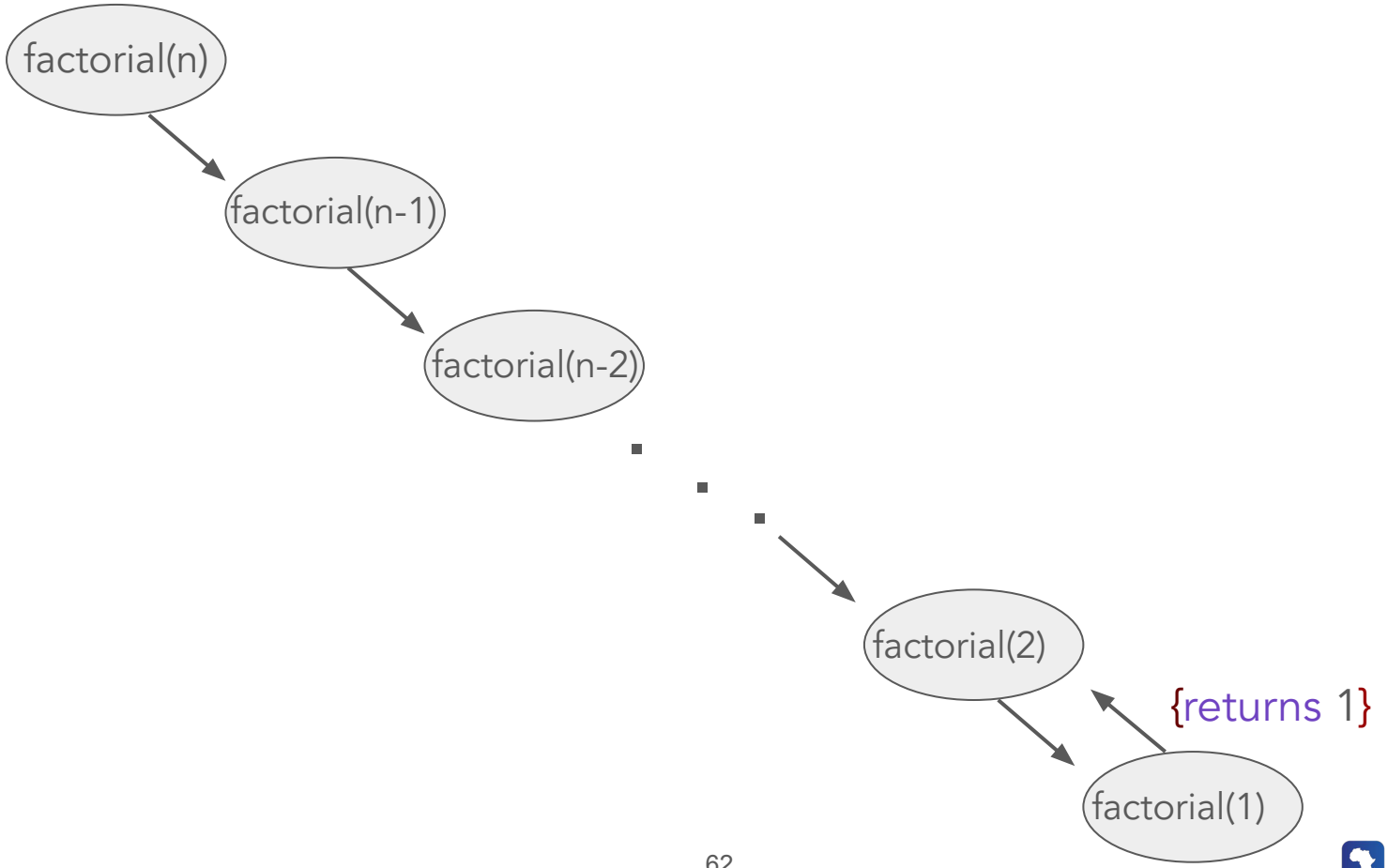

factorial(n)
factorial(n-1)
factorial(n-2)

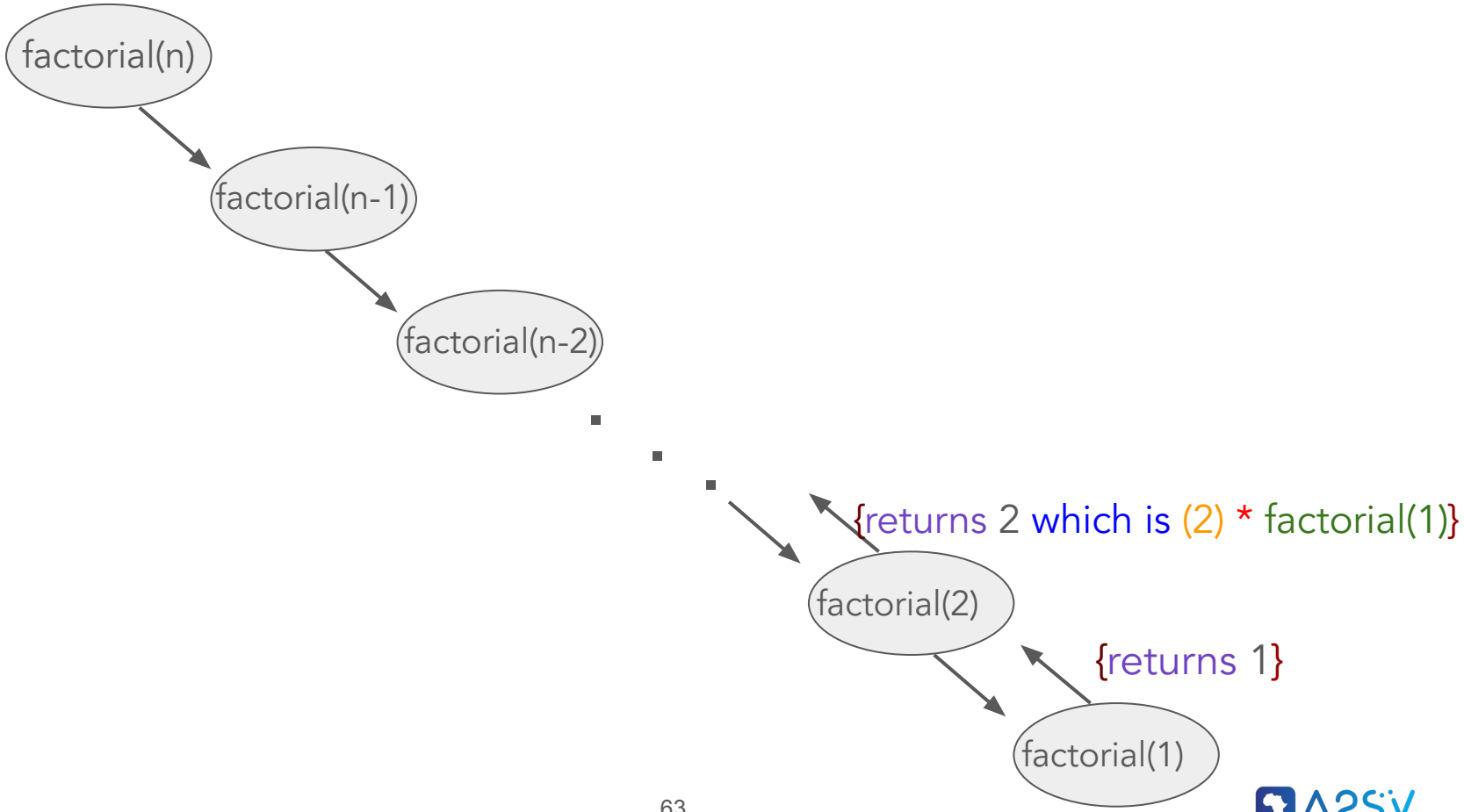

factorial(n)
factorial(n-1)
factorial(n-2)

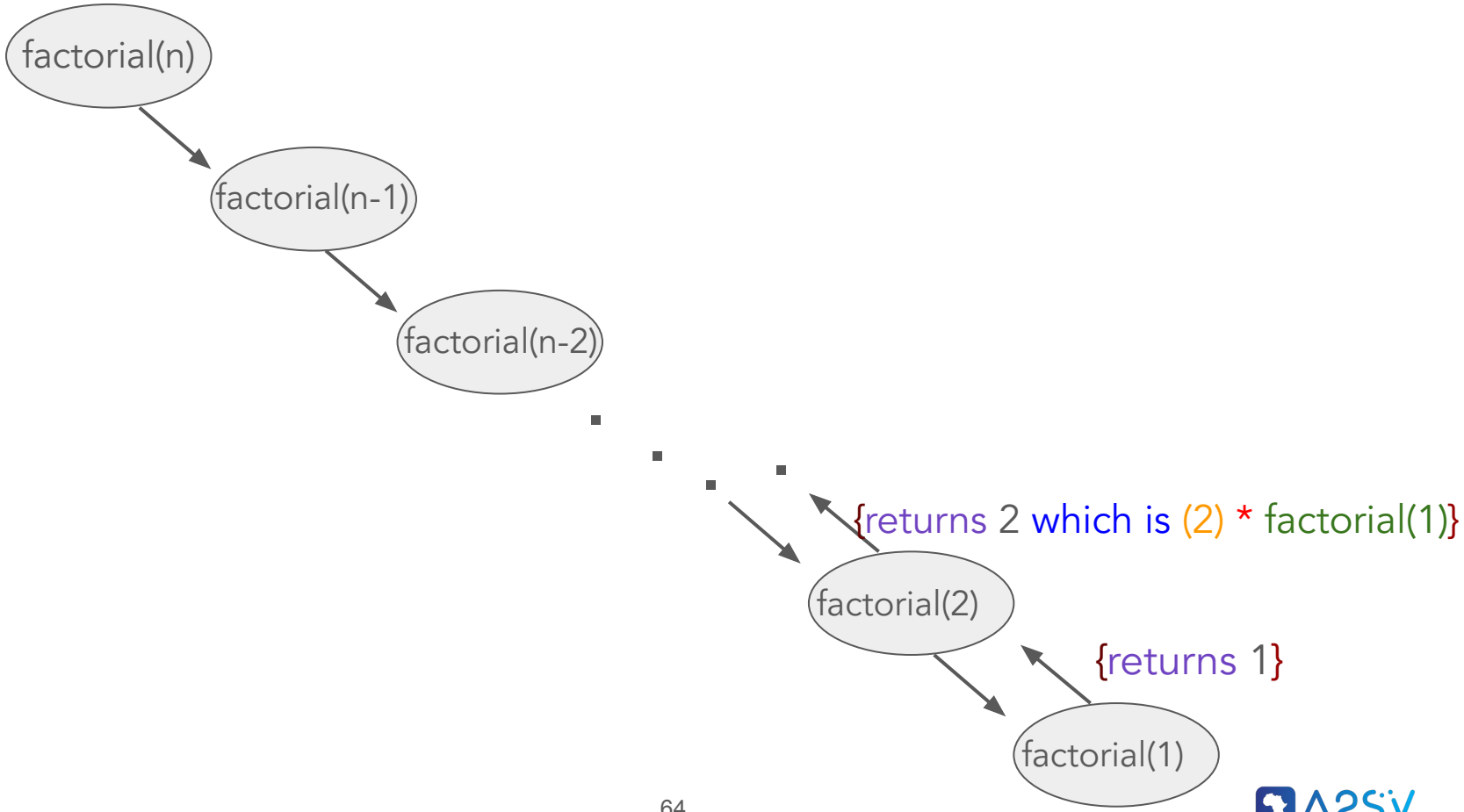

factorial(n)
factorial(n-1)
factorial(n-2)
calls factorial(2)
factorial(2)

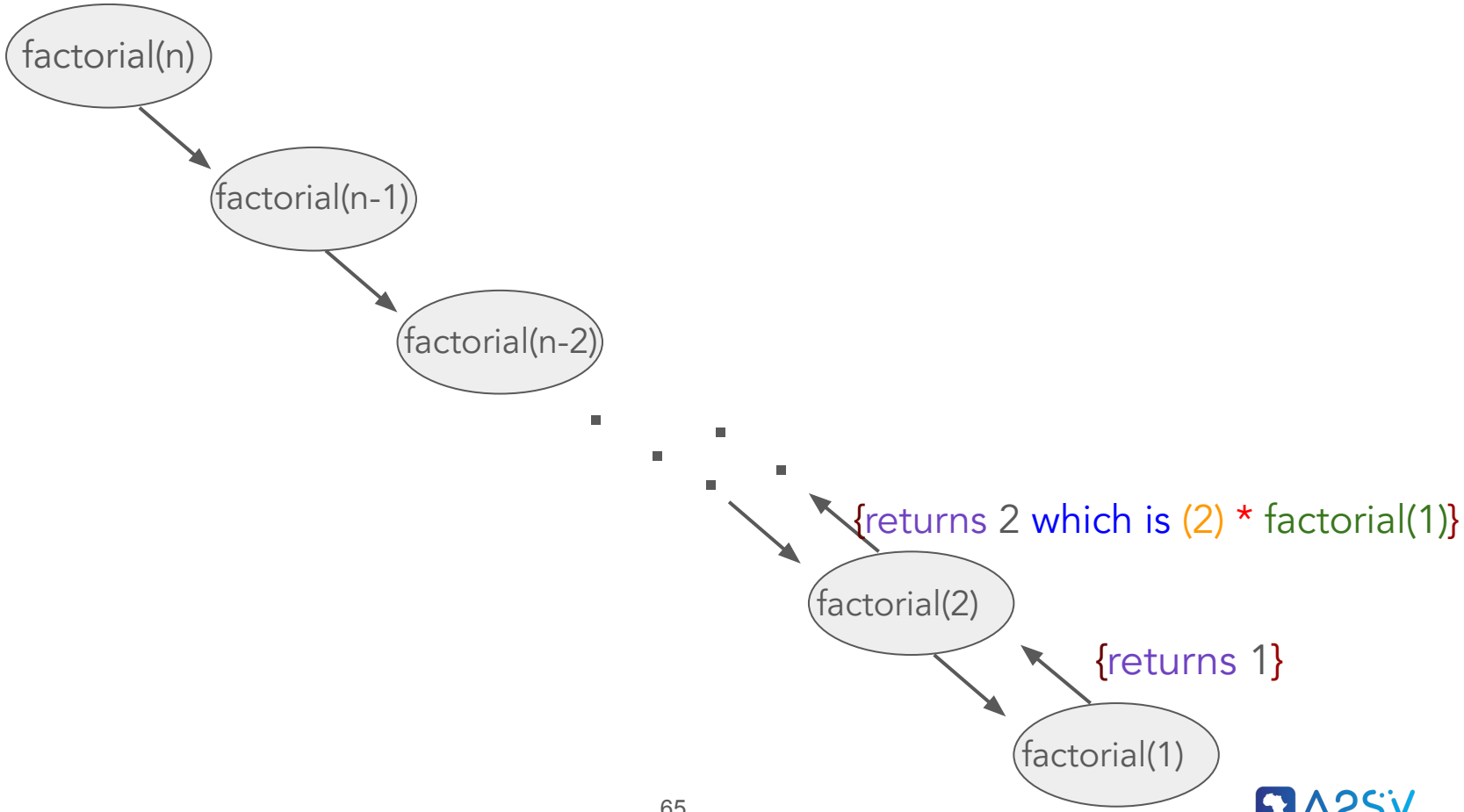

factorial(n)
factorial(n-1)
factorial(n-2)
factorial(2)
calls factorial(1)
factorial(1)

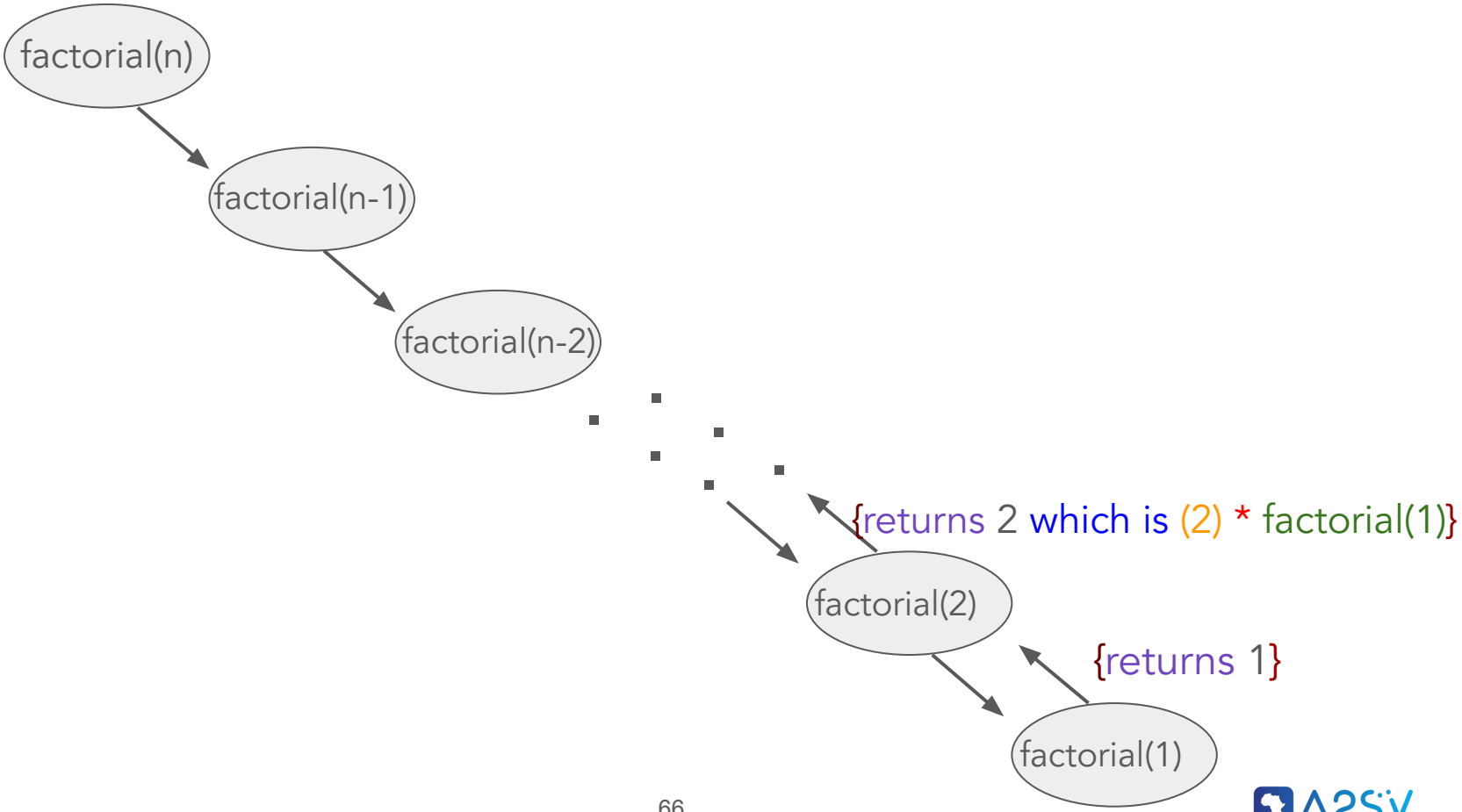

factorial(n)
factorial(n-1)
factorial(n-2)
factorial(2)
factorial(1)
{basecase}

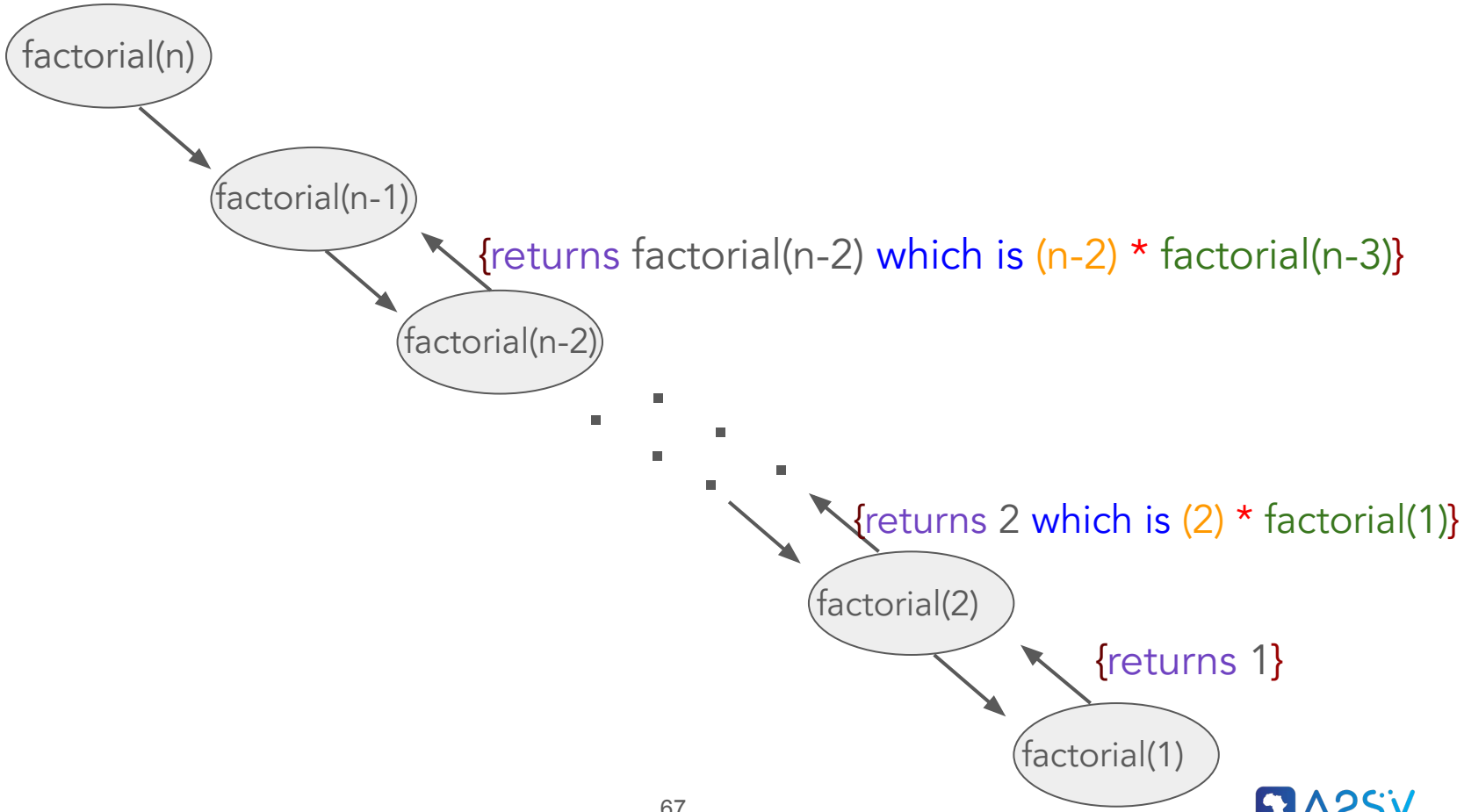

factorial(n)
factorial(n-1)
factorial(n-2)
factorial(2)
{returns 1}
factorial(1)

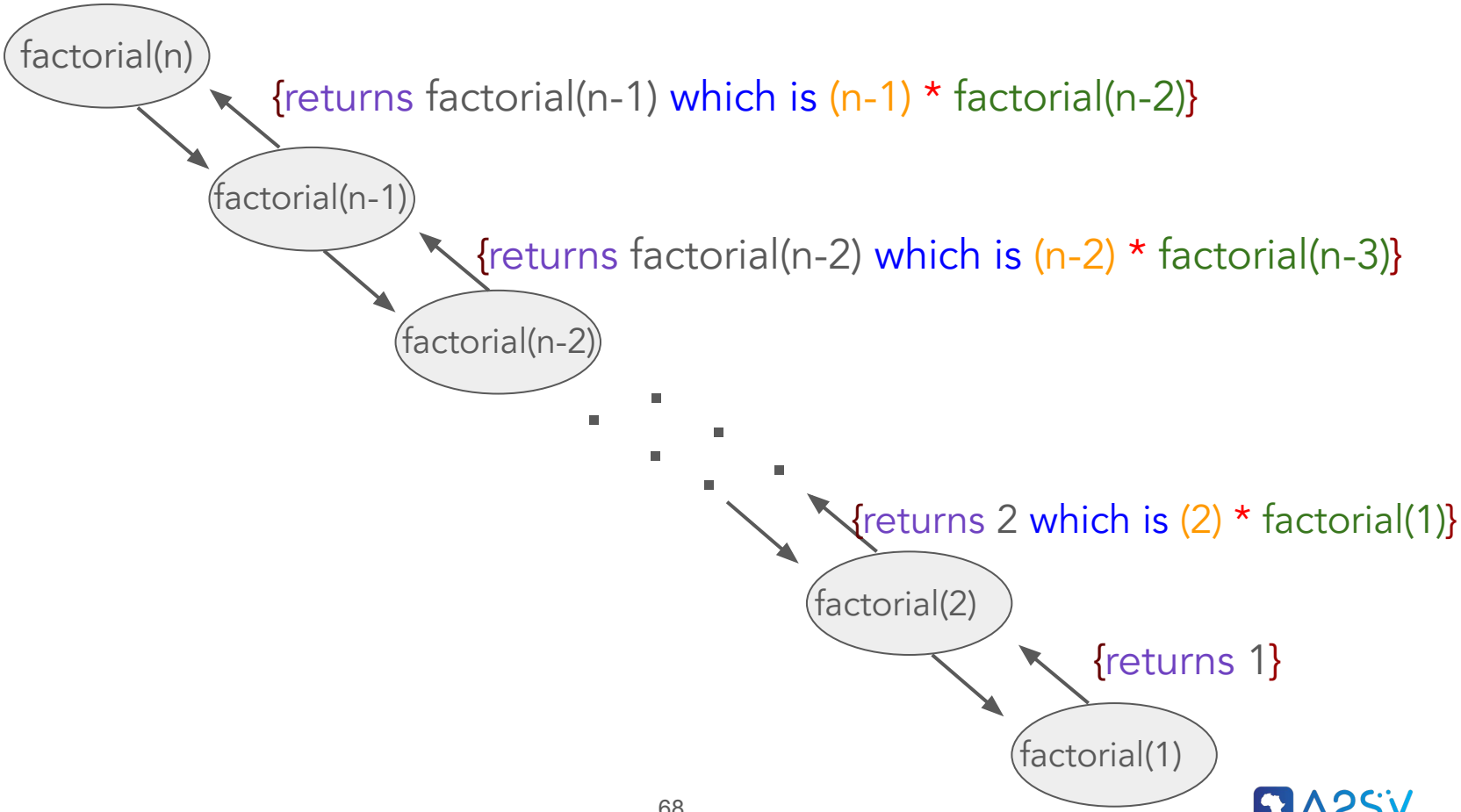

factorial(n)
factorial(n-1)
factorial(n-2)
{returns 2 which is (2) * factorial(1)}
factorial(2)
{returns 1}
factorial(1)

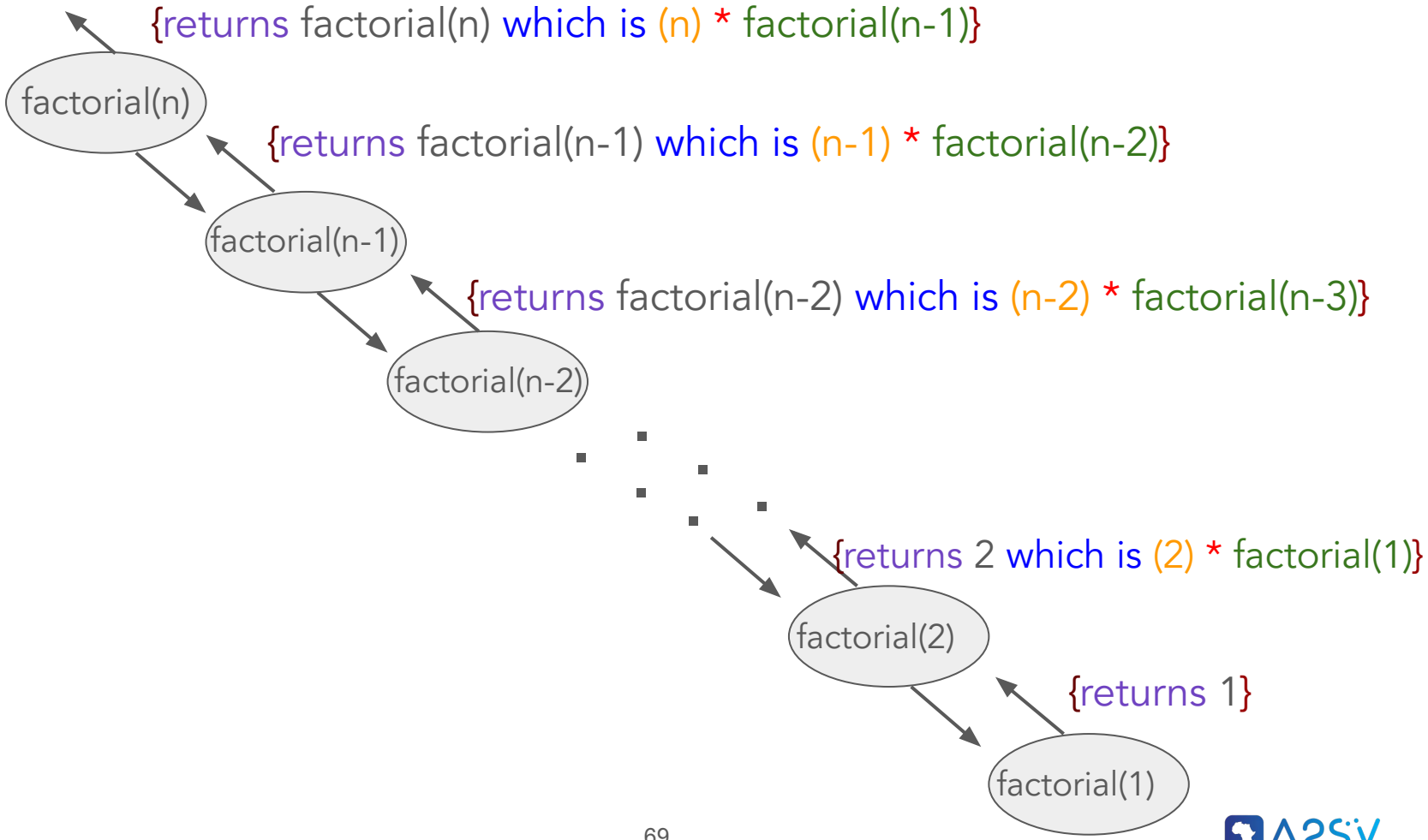

factorial(n)
factorial(n-1)
factorial(n-2)
{returns 2 which is (2) * factorial(1)}
factorial(2)
{returns 1}
factorial(1)


factorial(n)
factorial(n-1)
factorial(n-2)
{returns 2 which is (2) * factorial(1)}
factorial(2)
{returns 1}
factorial(1)


factorial(n)
factorial(n-1)
factorial(n-2)
{returns 2 which is (2) * factorial(1)}
factorial(2)
{returns 1}
factorial(1)


factorial(n)
factorial(n-1)
{returns factorial(n-2) which is (n-2) * factorial(n-3)}
factorial(n-2)
{returns 2 which is (2) * factorial(1)}
factorial(2)
{returns 1}
factorial(1)


factorial(n)
{returns factorial(n-1) which is (n-1) * factorial(n-2)}
factorial(n-1)
{returns factorial(n-2) which is (n-2) * factorial(n-3)}
factorial(n-2)
{returns 2 which is (2) * factorial(1)}
factorial(2)
{returns 1}
factorial(1)


{returns factorial(n) which is (n) * factorial(n-1)}
factorial(n)
{returns factorial(n-1) which is (n-1) * factorial(n-2)}
factorial(n-1)
{returns factorial(n-2) which is (n-2) * factorial(n-3)}
factorial(n-2)
{returns 2 which is (2) * factorial(1)}
factorial(2)
{returns 1}
factorial(1)

Pair Programming

Solution

Q1. What happens if we continue to divide a number which is a power of four by four?

- The final result will be 1

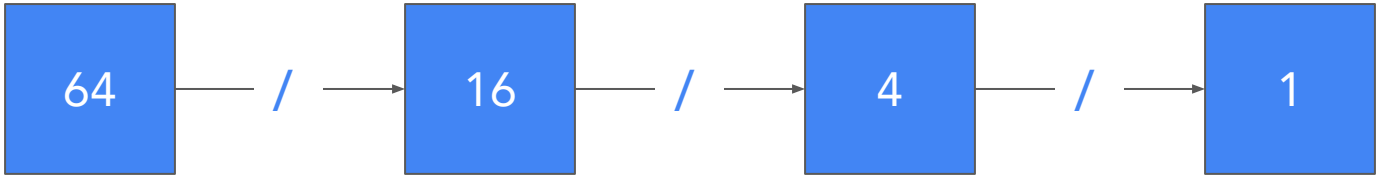
Q2. What if the number is not power of four?

- Their number will be below 1 eventually

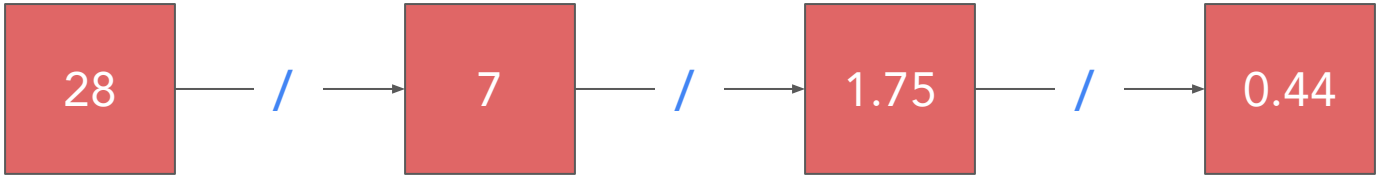
From the above questions we can see that we have two base cases and the state is the number that is divided by four every time we have recursive call.

A2SV

Dividing a power of four by four



Dividing a random number by four



Execution Tree

- Is a tree-like representation of all possible execution paths of a program.
- Each node in the tree represents a state of the program, and each edge represents a transition from one state to another.

Using Execution Tree

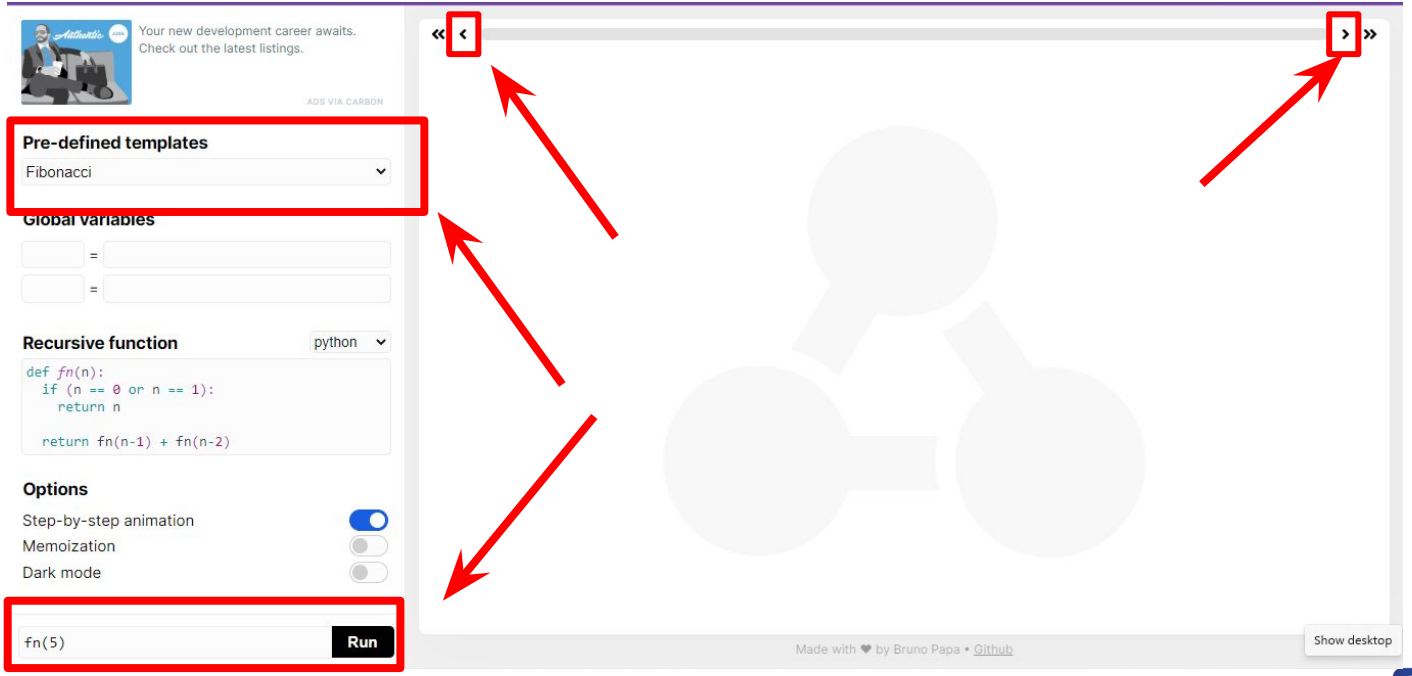
Fibonacci Sequence

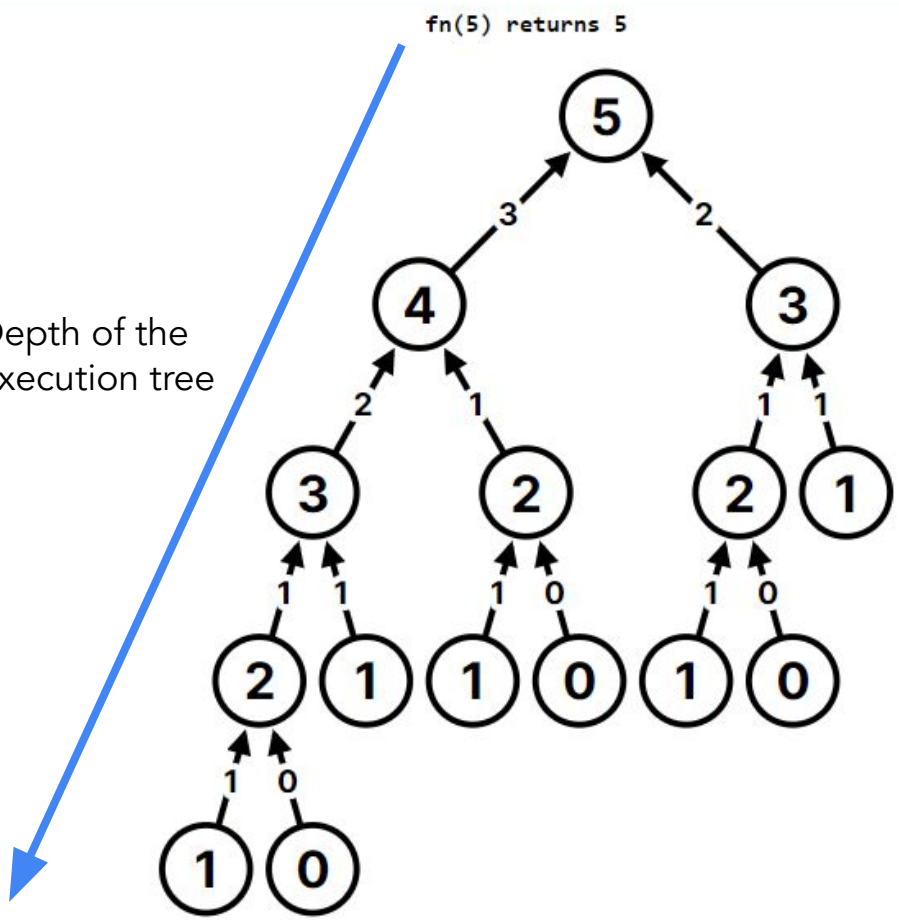
Fibonacci Sequence

- Series of numbers where each number is the sum of the two preceding numbers
- Let's use recursion to implement this.

Step by Step Simulation

Go to this LINK



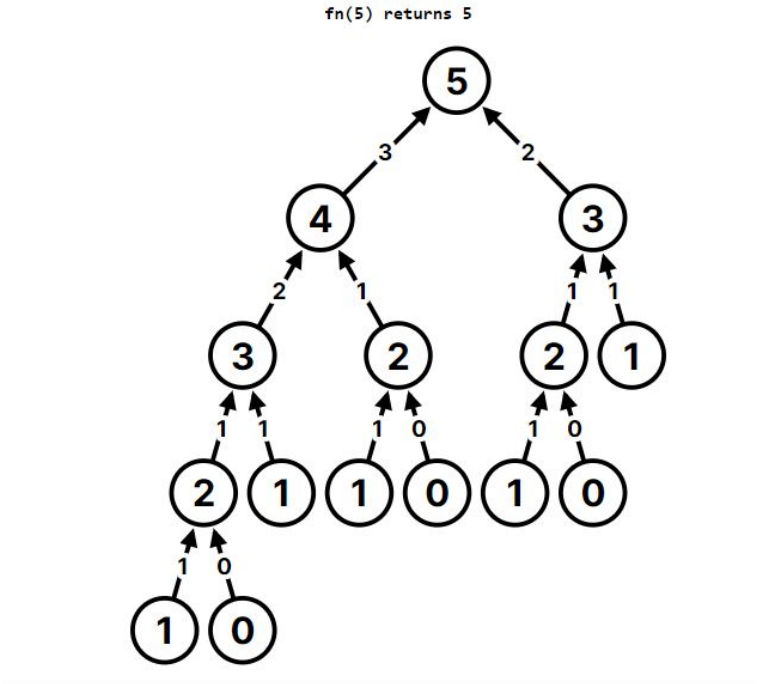


Depth of the execution tree

Num of nodes approximately : $2^{n+1} - 1$

Debugging using the execution tree

- Simulate the code using the execution tree using pen and paper by yourself.
- Simulate your code with recursion simulation sites
- Compare the results



How does recursion work?

How does the recursion work?

- The system uses some ordering to execute the calls
- The ordering is done based on Last In First Out order
- It uses stack

stack.top() -> currently executing

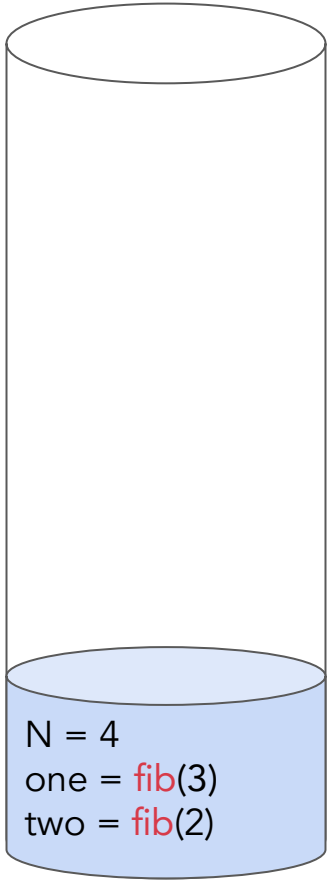
stack.push() -> calling function

stack.pop() -> returning a value

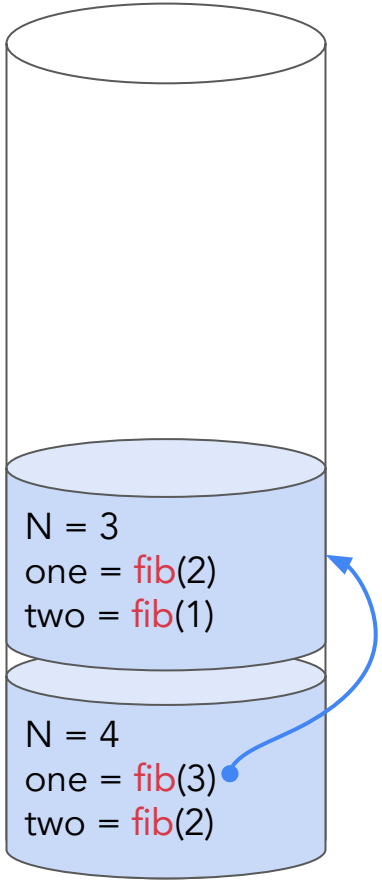
A2SV

Example Using Call Stack

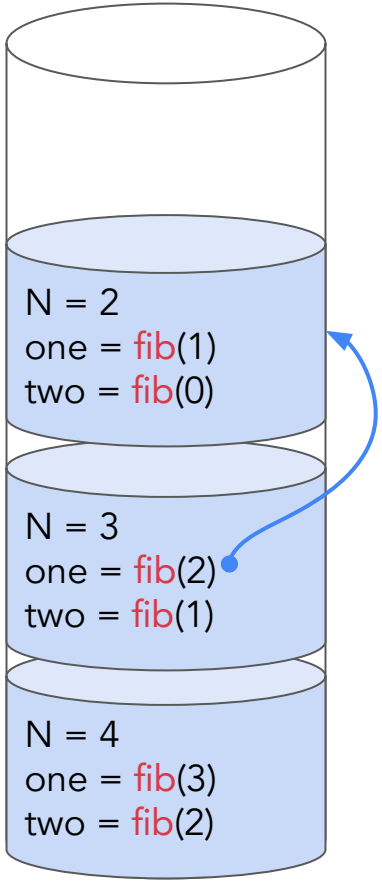
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



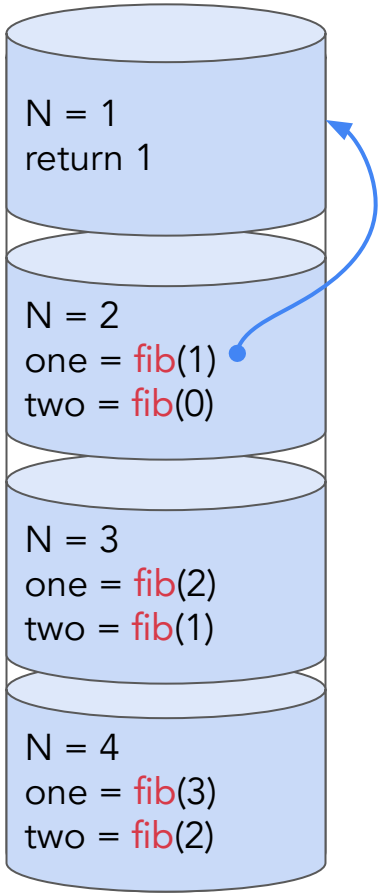
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



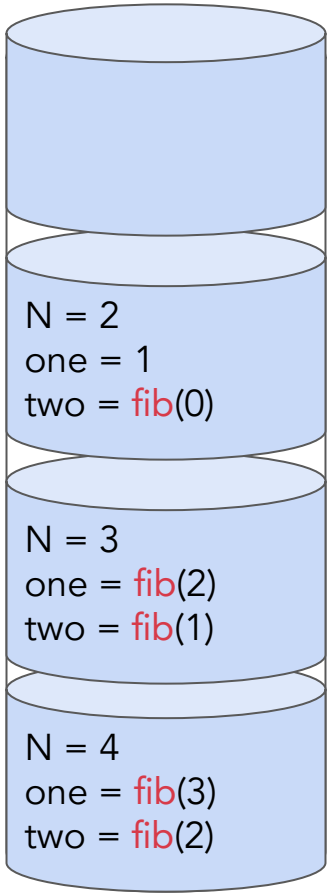
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



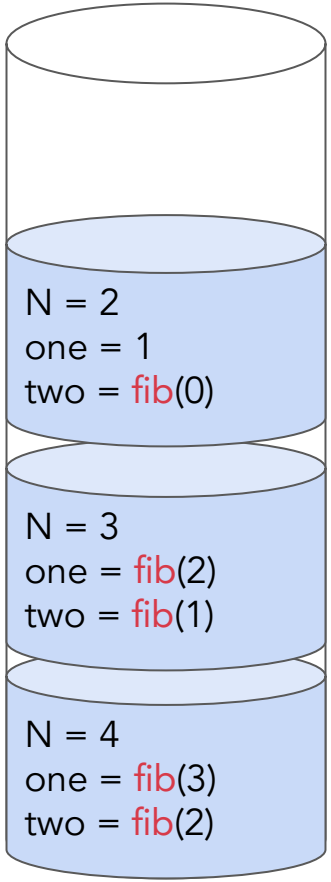
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



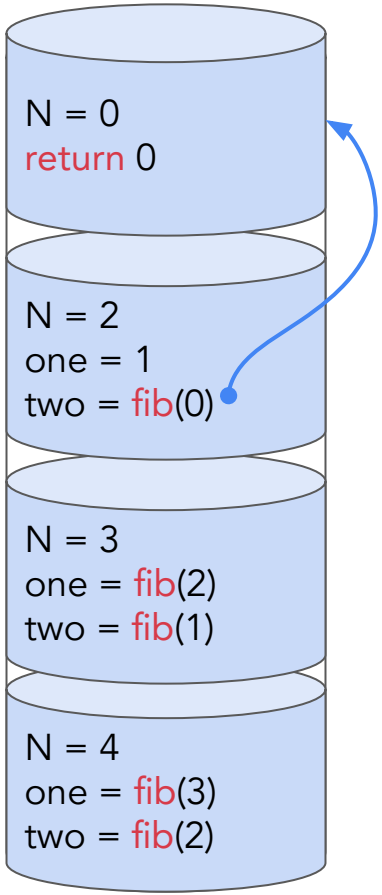
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



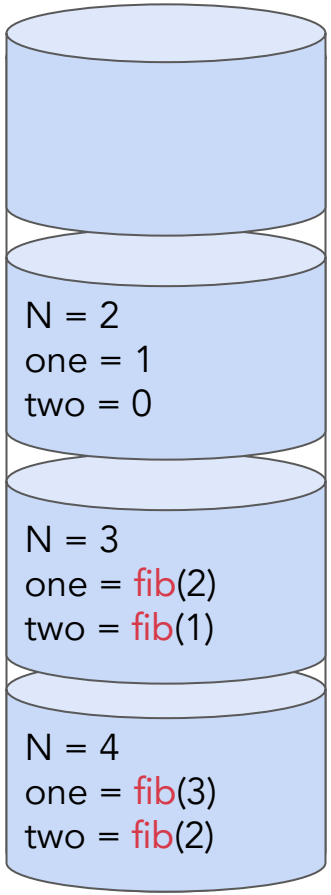
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



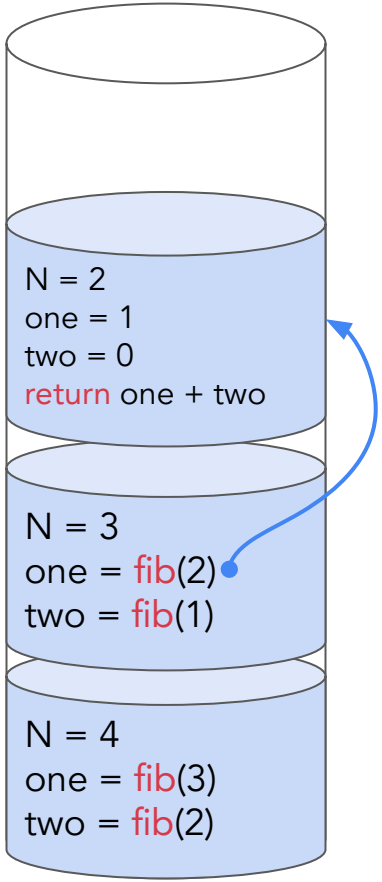
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



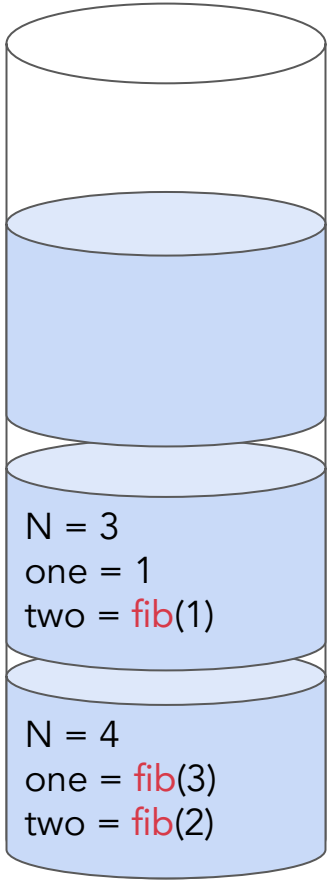
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



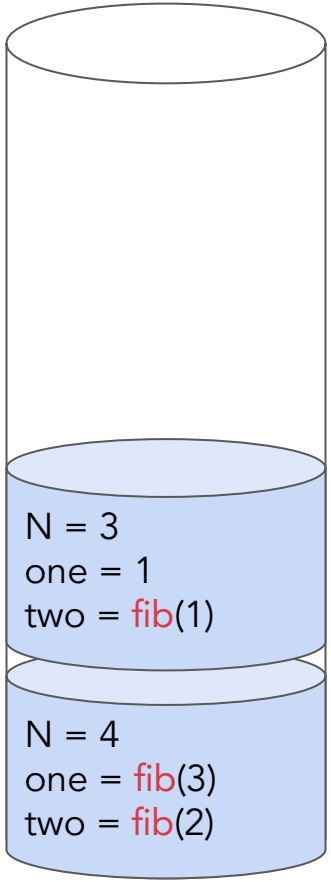
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



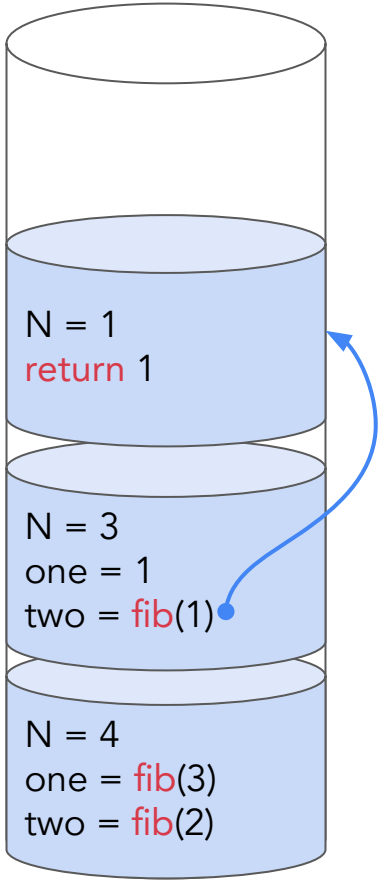
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



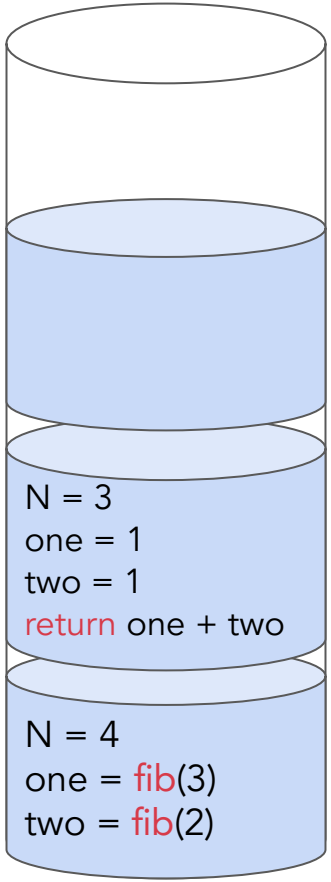
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



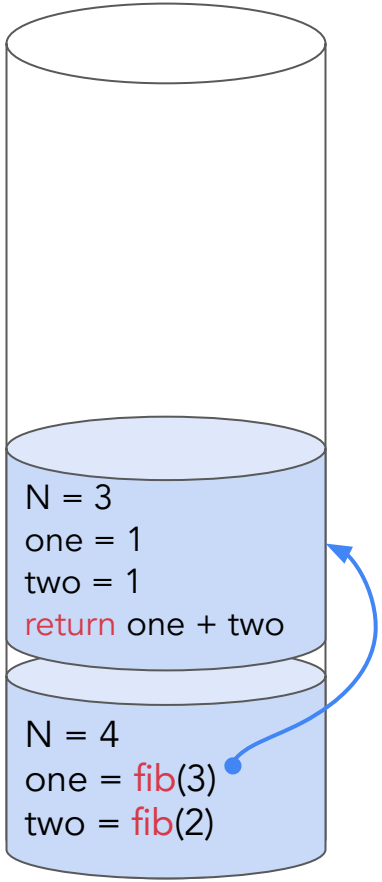
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



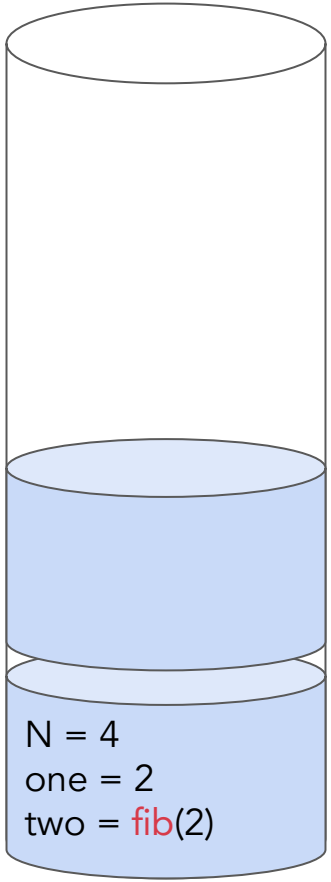
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



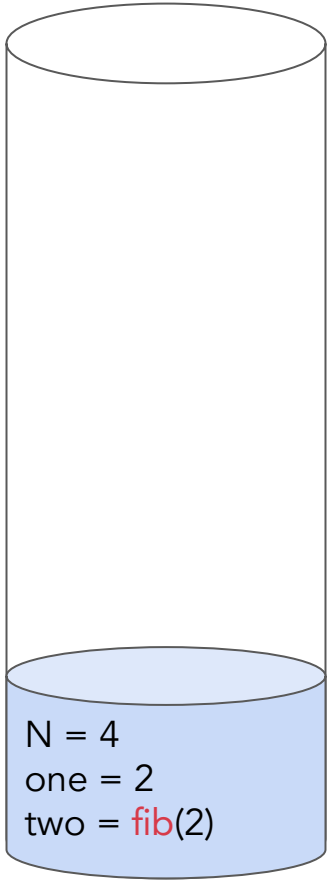
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



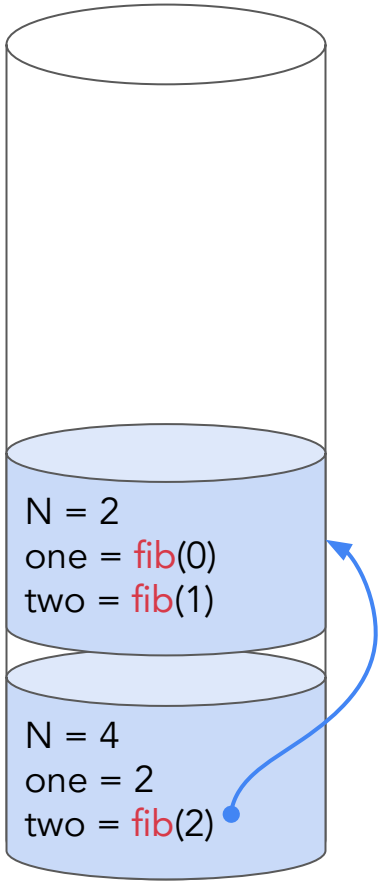
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



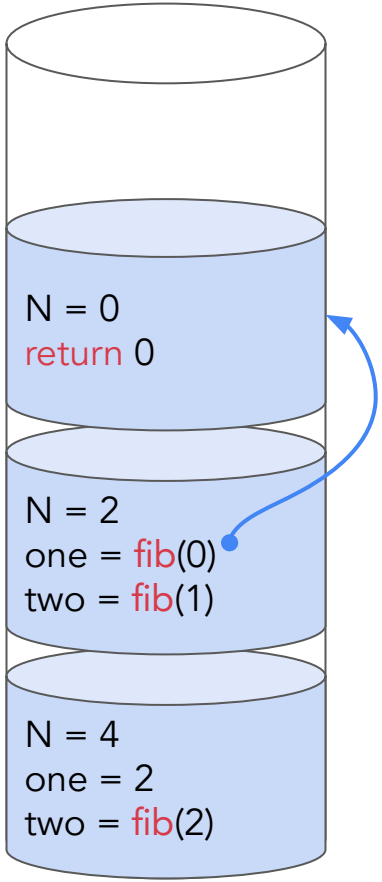
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



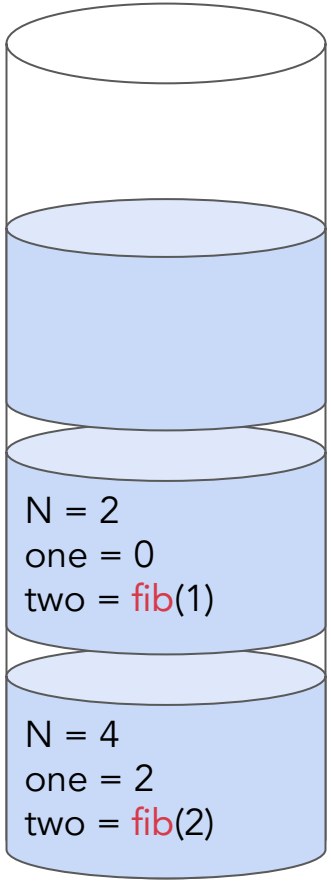
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



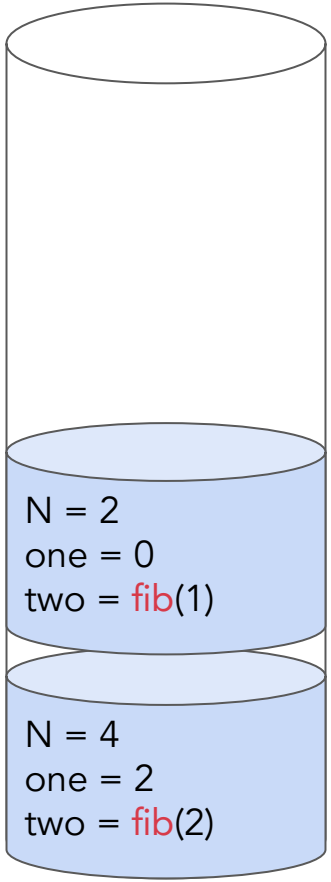
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



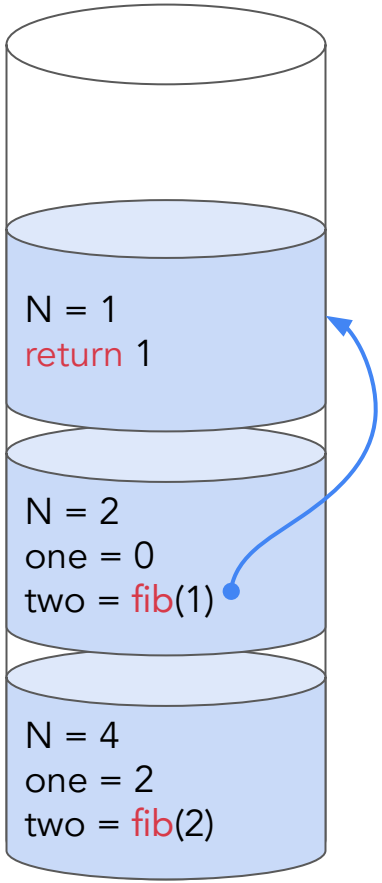
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



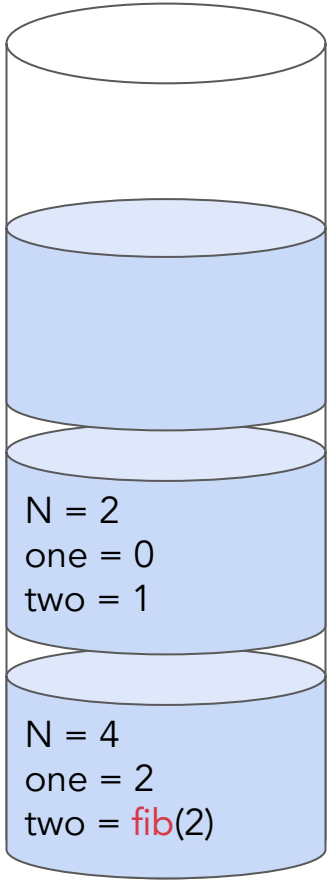
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



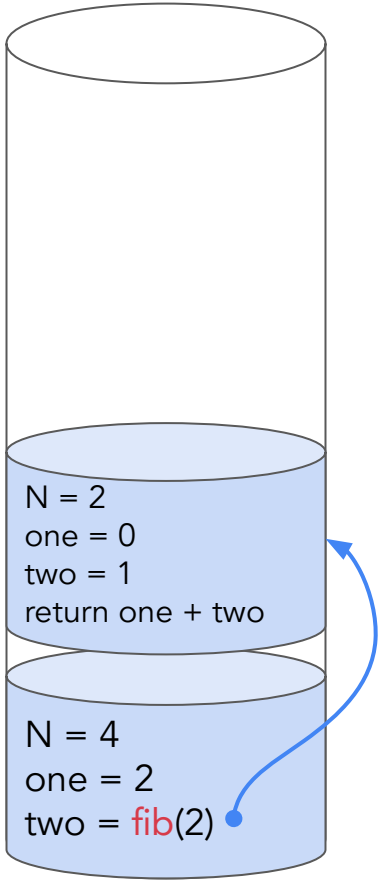
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



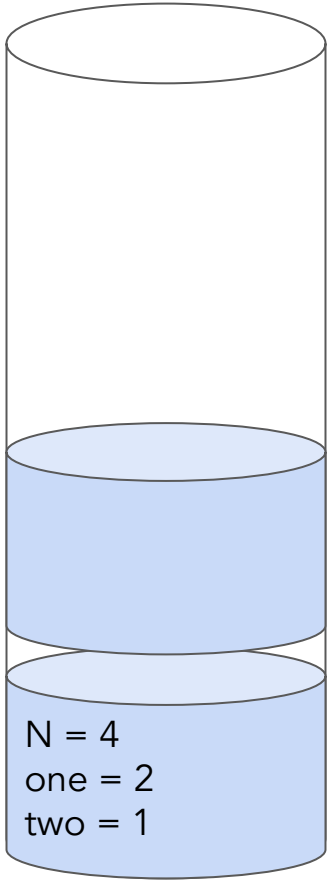
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



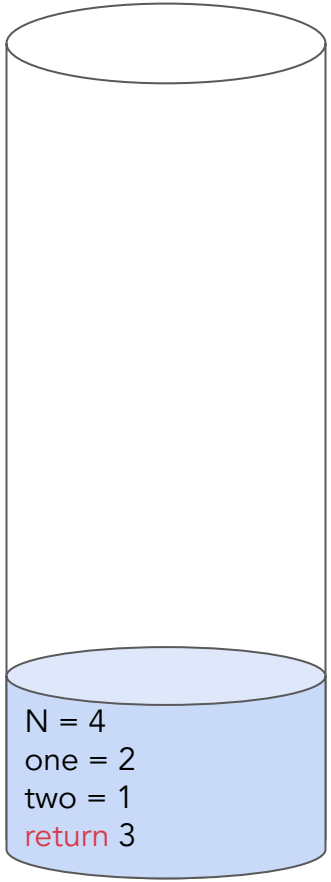
```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```

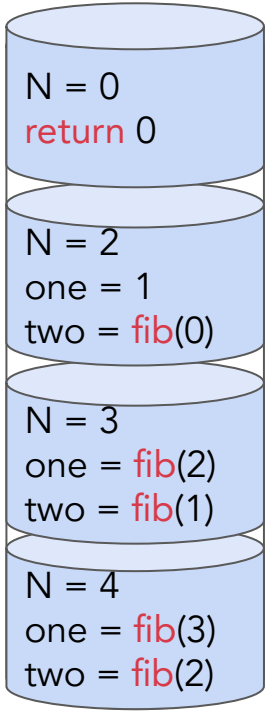


```python
def fib(n: int) -> int:
    if n == 1:
        return 1

    if n == 0:
        return 0

    one = fib(n-1)
    two = fib(n-2)

    return one + two
```



Debugging using the call stack and print statements

- Simulate the code using the call stack by yourself using pen and paper
- Use print statements to see if your code matches the simulation




```
def recursive_fn(state):
    print("state: ", state)

    .
    .
    .

    print("result:  ", result)
    return result
```

Complexity Analysis

Time Complexity
(Any suggestion?)

There are two parts

- Number of recursive calls
- The complexity of the function

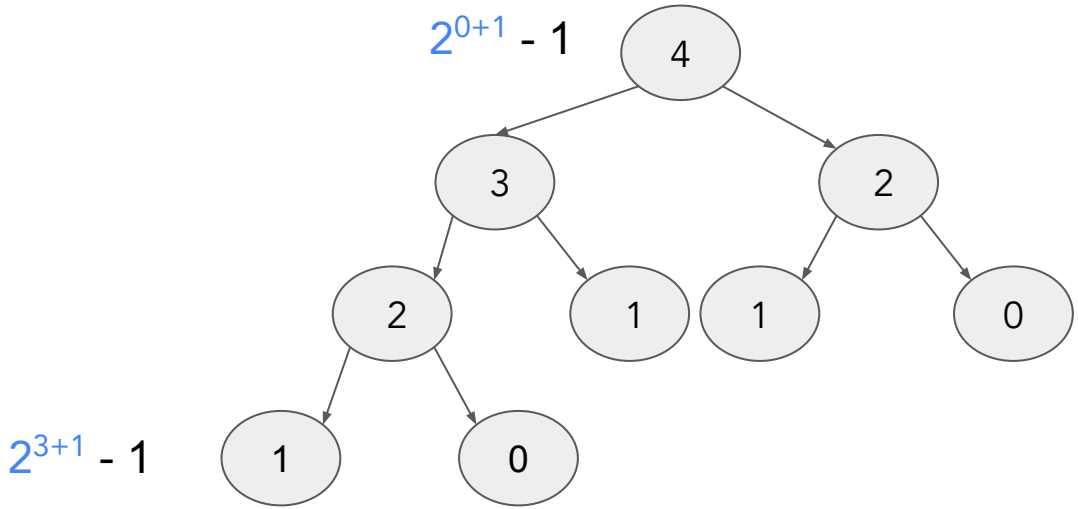
How do we find the number of calls?

- Draw the execution tree
- Execution tree is a tree that describes the flow of execution for the recursive function
- A node represents a call
- The total number of nodes is your final answer

Facts:

- Two branching on each node
- Height of the tree is equal to n
- In a full binary tree with n height, we have $2^{n+1} - 1$ nodes

Fib(4)



In general

- Let
  - b = number of branches on each noden (number of calls each function makes)
  - d = depth of the tree
- Total number of nodes = $b^{d+1} - 1$

- After only taking the dominant term

$$(\text{Num of branching})^{\text{depth of the tree}}$$

What is the complexity of the function?

- This is plain time and space complexity analysis of the inner working of the function
- Are we doing any costly operations in the function?

In Summary

$$\underbrace{O(\text{function})}_{\text{Cost of running function}} \bullet \underbrace{O((\text{Num of branching})^{\text{depth of the tree}})}_{\text{Number of recursive calls}}$$

Space Complexity

Before we discuss the space complexity, let's consider these questions

- How does the computer run recursion function?
- How does it know when to return and combine?

It uses call stack

What is the space complexity of the call stack?

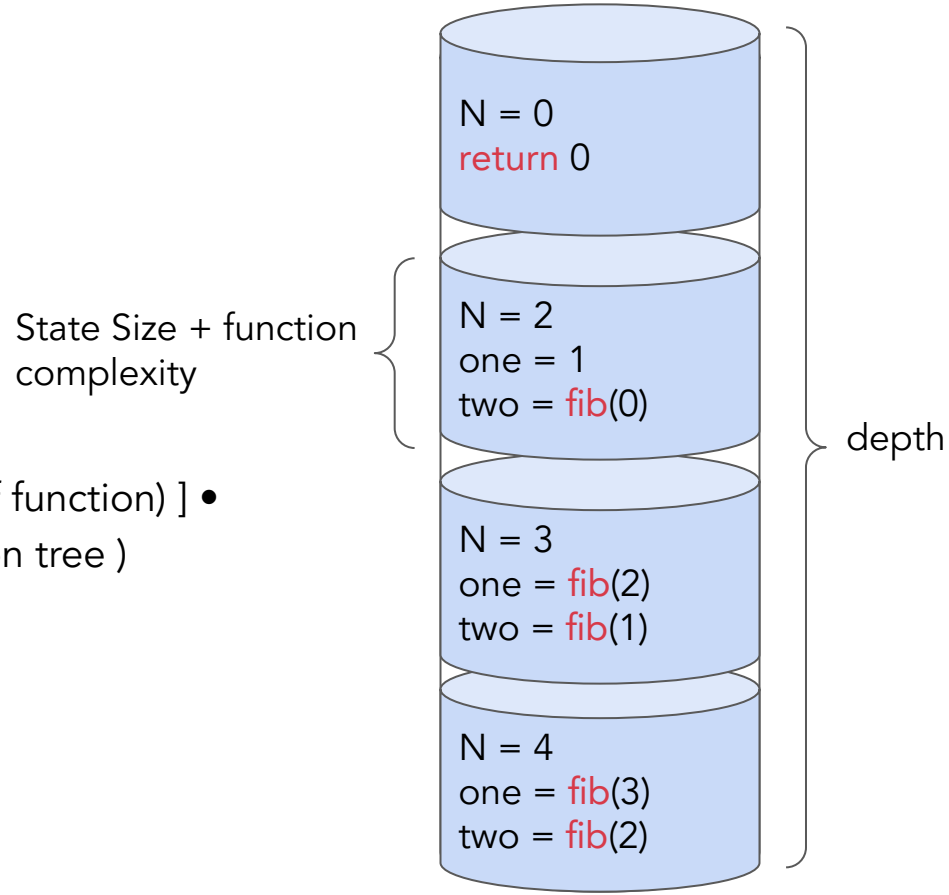
Maximum size of the stack which is maximum depth of the execution tree

Did we forget anything?

What about the states and space complexity of the function?

In Summary

[O(states' size) + O(space complexity of function)] •
O(maximum depth of the execution tree)



Checklist (Ask your instructor, if anything is missing)

- ❑ You have to be very comfortable with writing out the base case and recurrence relation of recursive function
- ❑ You have to be very comfortable with drawing execution tree of a recursion function
- ❑ Understanding the execution order using the execution tree
- ❑ Comfortably analyze the time and space complexity of a recursive function

Draw The Execution Tree

Pow(x,n)

Draw The Execution Tree (with two branches)

Pow(x,n)

Recursion versus Iteration

Anything that can be done recursively can be done iteratively

- Iteration is faster and more space-efficient than recursion.
- Recursion has function call overhead
- Iterative programs are easy to debug

So why do we even need recursion?

- If the solution to the given problem is achieved by breaking it down into its subproblems, the recursive approach is a more intuitive and natural way to solve it.

A2SV

Recognizing in Questions

- Look for patterns in the question that suggest that the same operation is being applied to smaller versions of the same problem.
- Look for base cases or stopping conditions that define when the recursion should end.
- Searching problems with branching conditions

A2SV

Common Pitfalls

Stack Overflow ?

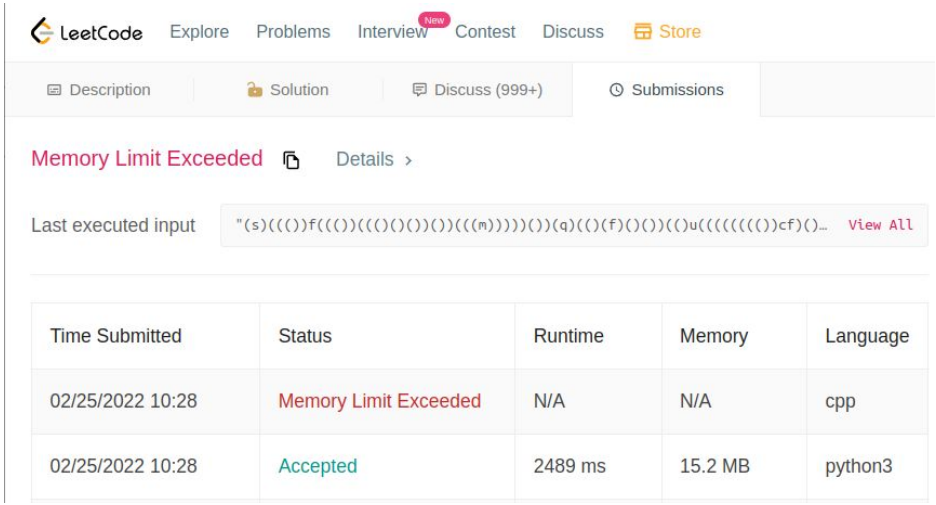
Occurs when the program uses up the memory assigned to it in the call stack

In the case of recursion we will mainly deal with two types:

- Memory Limit Exceeded
- Maximum Recursion Depth Exceeded.

Memory Limit Exceeded

is a type of stack overflow error that occurs when the parameters of our function take too much space.



Time Submitted	Status	Runtime	Memory	Language
02/25/2022 10:28	Memory Limit Exceeded	N/A	N/A	cpp
02/25/2022 10:28	Accepted	2489 ms	15.2 MB	python3

Maximum Recursion Depth Exceeded

is a type of stack overflow error that occurs when the number of recursive calls being stored in the call stack is greater than allowed.

Different programming languages have different maximum recursion Depth sizes:

- Python => 1000
- Javascript => 10,000
- Java => around 10,000
- C++ =>No Limit

```
>>> def count(number):
...     if not number:
...             return number
...     return 1 + count(number - 1)
...
>>> print(count(900))
900
>>> print(count(1000))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 4, in count
  File "<stdin>", line 4, in count
  File "<stdin>", line 4, in count
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
>>>
```

A function with no base case

```
def ask(person):

    behind = ask(person + 1)
    people = behind + 1

    return people
```

Learn to program
Make recursive function
No exit condition
Learn to program
Make recursive function
No exit condition
Learn to program
Make recursive function
No exit condition

A function with missing base case

```
def fib(n: int) -> int:
    if n == 1:
        return 1

    return fib(n-1) + fib(n-2)
```

What happens when n == 0?

Wrong recurrence relation

```
def numOfPeopleInQueue(n: int) -> int:
    if n == 1:
        return 1

    prev = numOfPeopleInQueue(n + 1) + 1
    return prev
```

Practice Questions

Power of Two
Power of three
Pascal's Triangle II
Remove Linked List Elements
Remove Nodes From Linked List
Merge Two Sorted List
Decode String
Reverse Linked List
Count Good Numbers
Flatten a Multilevel Doubly Linked List
Predict the Winner

Resources

- Leetcode Recursion I Card
- Leetcode Recursion II Card

Quote

"In order to understand recursion, one must first understand recursion."

Unknown